



`alittlebyte.swe@gmail.com`

Norme di Progetto

Gruppo	aLittleByte (gruppo 19)
Versione	2.0.0
Redattori	Eleonora Bellet, Diego Belluz, Alex Oloni, Angelica Gastaldello, Angela Maule, Leonardo Soligo, V.Y. Kaneda Fernandes
Verificatori	V.Y. Kaneda Fernandes, Eleonora Bellet, Angela Maule, Alex Oloni, Diego Belluz, Angelica Gastaldello
Destinatari	Prof. Tullio Vardanega, Prof. Riccardo Cardin
Data	08/09/2026

Registro delle Modifiche

Version	Data	Descrizione	Redattore	Verificatore	Approvatore
2.0.0	08/09/2026	Approvazione versione 2.0.0	-	-	Leonardo Soligo
1.12.0	08/09/2026	Ristrutturazione delle sezioni dell'architettura dei processi secondo uno schema uniforme e in stile procedurale	Angelica Gastaldello	Alex Oloni	-
1.11.0	05/09/2026	Aggiunta sezioni accertamento della qualità, infrastruttura, formazione e miglioramento dei processi	Leonardo Soligo	Angela Maule	-
1.10.0	02/09/2026	Aggiunta sezioni stato di un documento e convenzioni di versionamento	Eleonora Bellet	Diego Belluz	-
1.9.0	25/08/2026	Aggiornamento e correzione di errori testuali	Alex Oloni	Eleonora Bellet	-
1.8.0	21/08/2026	Aggiunta sezione applicazione del tailoring	Leonardo Soligo	Angela Maule	-
1.7.0	14/08/2026	Aggiunta della redazione di Specifica Tecnica e Manuale Utente in fase PB a cura del Progettista	Angelica Gastaldello	Alex Oloni	-
1.6.0	11/08/2026	Aggiunta sezioni test di sistema e test di regressione	Angelica Gastaldello	Diego Belluz	-
1.5.0	05/08/2026	Aggiunta sezioni test di unità e test di integrazione	Leonardo Soligo	Angela Maule	-
1.4.0	29/07/2026	Aggiunta sezione analisi statica automatica	Angela Maule	Diego Belluz	-
1.3.0	23/07/2026	Aggiunta sezione automazione e rilascio continuo (CI/CD)	Angela Maule	Alex Oloni	-
1.2.0	17/07/2026	Aggiunta sezione nomenclatura dei branch e alberatura della repository	Angela Maule	Angelica Gastaldello	-
1.1.0	14/07/2026	Aggiunta sezione tracciamento delle ore	V.Y. Kaneda Fernandes	Eleonora Bellet	-
1.0.0	28/05/2026	Approvazione versione 1.0.0	-	-	Angelica Gastaldello
0.15.0	27/05/2026	Aggiunta sezione processo di miglioramento continuo	Eleonora Bellet	V.Y. Kaneda Fernandes	-
0.14.0	20/05/2026	Aggiunta sezione metriche	Diego Belluz	V.Y. Kaneda Fernandes	-

Version	Data	Descrizione	Redattore	Verificatore	Approvatore
0.13.0	18/05/2026	Aggiunta sezione qualità del software	Diego Belluz	Eleonora Bellet	-
0.12.0	8/05/2026	Struttura dei diari di bordo	Diego Belluz	Eleonora Bellet	-
0.11.2	7/05/2026	Corretti errori di sintassi	Diego Belluz	V.Y. Kaneda Fernandes	-
0.11.1	22/04/2026	Aggiornate e corrette alcune sezioni delle norme	Alex Oloni	Eleonora Bellet	-
0.11.0	19/04/2026	Tracciamento delle azioni, comunicazione, riunioni	Eleonora Bellet	V.Y. Kaneda Fernandes	-
0.10.0	18/04/2026	Processi Organizzativi (Definizione e scopo, gestione degli sprint, ruoli)	Alex Oloni	Eleonora Bellet	-
0.9.0	17/04/2026	Aggiunta sezione Validazione e Processo di Validazione	Eleonora Bellet	V.Y. Kaneda Fernandes	-
0.8.0	16/04/2026	Controllo della configurazione, Stato della configurazione, Attività di Verifica, Analisi Statica	Alex Oloni	Eleonora Bellet	-
0.7.0	14/04/2026	Denominazione dei documenti, Produzione e Gestione della configurazione	Eleonora Bellet	V.Y. Kaneda Fernandes	-
0.6.0	11/04/2026	Processi di supporto (scopo, strumenti utilizzati, documenti prodotti, struttura dei documenti)	Diego Belluz	Angela Maule	-
0.5.0	9/04/2026	Processi di sviluppo	Eleonora Bellet	V.Y. Kaneda Fernandes	-
0.4.0	8/04/2026	Processi di fornitura	Diego Belluz	Angela Maule	-
0.3.0	4/04/2026	Architettura dei processi	Eleonora Bellet	V.Y. Kaneda Fernandes	-
0.2.0	31/03/2026	Scopo del documento e scopo del prodotto	Diego Belluz	Angela Maule	-
0.1.0	31/03/2026	Struttura del documento	Eleonora Bellet	V.Y. Kaneda Fernandes	-

Indice

Indice

1 Introduzione

1.1 Scopo del documento

Il presente documento ha l'obiettivo di definire e regolamentare il Way of Working (WoW)^G adottato dal gruppo *aLittleByte* per la realizzazione del progetto didattico. Al suo interno sono specificate le procedure operative, gli strumenti adottati e le regole di collaborazione interne al team, al fine di garantire un approccio strutturato, efficiente e tracciabile per l'intero ciclo di vita del progetto Nexum^G.

La stesura del seguente documento segue un approccio incrementale^G: le norme qui descritte verranno costantemente aggiornate e affinate nel tempo, supportando il miglioramento continuo dei processi organizzativi, di sviluppo e di supporto del gruppo.

1.2 Scopo del prodotto

Il prodotto da realizzare consiste in una piattaforma software *stand-alone* sviluppata su richiesta dell'azienda proponente^G *Eggon*, che si affianca a Nexum^G — piattaforma digitale dedicata alla comunicazione interna e alla gestione HR — comunicando con essa tramite API^G. Il sistema è architetturealmente indipendente da Nexum^G e comprende due moduli basati sull'Intelligenza Artificiale^G:

- **AI Assistant Generativo**^G: Un modulo dedicato agli utenti amministrativi (dashboard^G) che permette la creazione autonoma di contenuti aziendali (titolo, testo, immagini di copertina) a partire da un prompt^G, garantendo l'adeguamento automatico del tono e dello stile del messaggio.
- **AI Co-Pilot per Consulenti del Lavoro (CdL)**^G: Un sistema avanzato in grado di riconoscere automaticamente la tipologia di documenti caricati (es. cedolini, comunicazioni), estrarne i destinatari tramite tecniche di OCR^G ed Entity Resolution^G, e automatizzare lo smistamento e il dispaccio^G massivo.

L'obiettivo finale è migliorare l'efficienza dei processi HR e semplificare l'interoperabilità tra le aziende e gli studi dei Consulenti del Lavoro. Per fare ciò il gruppo *aLittleByte* si impegna a completare il progetto entro la data fissata 10 Settembre 2026, con un costo totale di budget di 12.575 €.

Per ulteriori informazioni si rimanda al documento Analisi dei Requisiti^G.

1.3 Glossario

Al fine di evitare ambiguità e garantire una comprensione uniforme della terminologia utilizzata, è stato redatto un documento esterno denominato Glossario^G. I termini tecnici, gli acronimi e le parole con un significato specifico all'interno del progetto sono contrassegnati nel testo da una "G" in apice (es. parola^G). La loro definizione completa è consultabile nel Glossario^G.

1.4 Riferimenti

1.4.1 Normativi

- **Regolamento del Progetto Didattico:** Anno accademico 2025/2026, Corso di Ingegneria del Software, Università di Padova
 - <https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>
 - *Data Ultimo accesso: 02/09/2026*
- **Capitolato^G d'appalto C5:** Nexum^G - Eggon S.r.l.
 - <https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C5.pdf>
 - *Data Ultimo accesso: 05/09/2026*

1.4.2 Informativi

- **Glossario^G v3.0.0:** Documento interno del progetto contenente la definizione dei termini tecnici, di dominio e degli acronimi utilizzati nella documentazione.
 - <https://alittlebyte-19.github.io/Documentazione/glossario.html>
 - *Data Ultimo accesso: 08/09/2026*

2 Architettura dei processi

Per la realizzazione di un prodotto di qualità che soddisfi le esigenze del cliente e degli utenti, il gruppo *aLittleByte* adotta la classificazione dei processi di ciclo di vita prevista dallo standard ISO/IEC 12207, suddividendoli in tre macro-aree:

- **Processi primari:** i processi che concorrono direttamente alla realizzazione del prodotto, ovvero fornitura e sviluppo.
- **Processi di supporto:** i processi che sostengono i processi primari garantendone la qualità e la tracciabilità, ovvero documentazione, gestione della configurazione, accertamento della qualità, verifica e validazione.
- **Processi organizzativi:** i processi che governano il modo in cui il gruppo lavora, ovvero gestione dei processi, infrastruttura, formazione e miglioramento dei processi.

Per rendere il presente capitolo effettivamente utilizzabile durante lo svolgimento delle attività, ogni processo è normato secondo il medesimo schema espositivo:

- **Scopo:** il risultato che il processo deve produrre e la ragione per cui il gruppo lo applica.
- **Strumenti a supporto:** gli strumenti che devono essere impiegati nello svolgimento del processo.
- **Attività previste:** l'elenco delle attività in cui il processo si articola.

- **Procedure operative:** per ciascuna attività, la sequenza numerata dei passi da eseguire, con indicazione del ruolo responsabile di ogni passo.

All'interno delle procedure, il ruolo indicato all'inizio di ogni passo è il responsabile della sua esecuzione. Le formulazioni con “deve” individuano obblighi vincolanti per tutti i membri del gruppo, mentre quelle con “può” individuano facoltà il cui esercizio è rimesso al giudizio del ruolo indicato.

2.1 Processi primari

In questa sezione viene descritto come nasce il software. I processi primari comprendono tutte le attività direttamente coinvolte nella creazione del prodotto e si articolano in fornitura e sviluppo.

2.1.1 Fornitura

2.1.1.1 Scopo Il processo di fornitura deve trasformare le necessità espresse nel capitolato^G C5 in un insieme di requisiti tecnici e di vincoli operativi concordati con la proponente^G *Eggon*, stabilendo il perimetro del lavoro, il budget e le scadenze entro cui il gruppo si impegna a consegnare il prodotto.

2.1.1.2 Strumenti a supporto

- **GitHub**^G: per il versionamento della documentazione di fornitura e per la conversione di ogni impegno assunto in una *Issue*^G assegnata.
- **LaTeX**^G: per la redazione dei documenti contrattuali e di pianificazione.
- **Posta elettronica e Telegram**: per le comunicazioni formali e per i chiarimenti rapidi con il referente aziendale, secondo le regole definite nel paragrafo sulla comunicazione.
- **Google Meet**: per gli incontri periodici con la proponente^G.

2.1.1.3 Attività previste

1. Analisi e valutazione del capitolato^G.
2. Stima e pianificazione^G delle risorse.
3. Negoziazione e definizione dell'MVP^G.
4. Produzione della documentazione di fornitura.

2.1.1.4 Analisi e valutazione del capitolato

1. Ogni membro del gruppo studia i capitolati disponibili e ne valuta i rischi tecnologici, le opportunità di apprendimento e la coerenza con le competenze del team.
2. Il gruppo raccoglie gli esiti delle valutazioni individuali nel documento di *Analisi dei Capitolati*, confrontando le alternative sulla base dei medesimi criteri.
3. Il Responsabile^G organizza un colloquio con ciascuna azienda proponente^G dei capitolati ritenuti idonei e ne fa redigere il verbale esterno^G.
4. Il gruppo delibera la scelta del capitolato^G in riunione interna e la motivazione della scelta deve essere riportata nel relativo verbale interno^G.
5. Il Responsabile^G formalizza la candidatura inviando la *Lettera di Candidatura* all'indirizzo del corpo docente.

2.1.1.5 Stima e pianificazione delle risorse

1. Il Responsabile^G scompone il lavoro complessivo nelle fasi delimitate dalle milestone^G RTB^G e PB^G, fissando come termine di consegna il 10 settembre 2026.
2. Il Responsabile^G definisce il preventivo^G di periodo nel Piano di Progetto^G, ripartendo il budget complessivo di 12.575 € tra i ruoli e le iterazioni previste.
3. Il Responsabile^G suddivide ogni fase in Sprint^G di durata fissa e ne pianifica il contenuto secondo la procedura di pianificazione^G definita nel processo di gestione dei processi.
4. Al termine di ogni Sprint^G il Responsabile^G aggiorna il consuntivo^G e le previsioni di costo a finire, motivando nel Piano di Progetto^G gli scostamenti rilevati.

2.1.1.6 Negoziazione e definizione dell'MVP

1. L'Analista^G raccoglie le richieste della proponente^G durante gli incontri esterni e le formalizza come requisiti nell'Analisi dei Requisiti^G.
2. L'Analista^G classifica ogni requisito per tipologia e per urgenza, distinguendo i requisiti funzionali^G obbligatori da quelli opzionali^G.
3. Il gruppo concorda con *Eggon* il perimetro del Minimum Viable Product^G, verificando che le funzionalità core (AI Assistant Generativo^G e AI Co-Pilot^G) siano incluse e prioritarie.
4. Ogni variazione del perimetro concordata dopo l'accordo iniziale deve essere verbalizzata nel verbale esterno^G dell'incontro in cui è stata discussa e recepita nell'Analisi dei Requisiti^G entro l'iterazione successiva.

2.1.1.7 Produzione della documentazione di fornitura La documentazione è lo strumento principale con cui il gruppo garantisce trasparenza verso i committenti e coerenza interna. I documenti che il processo di fornitura deve produrre e mantenere aggiornati sono i seguenti:

- **Analisi dei Requisiti^G**: formalizza le funzionalità del sistema. Include la modellazione degli attori (Redattore^G, Analista dati^G, Consulente del lavoro^G), la descrizione dei casi d'uso (UC^G) e la classificazione dei requisiti funzionali^G, qualitativi e di vincolo.
- **Norme di Progetto^G**: definiscono il regolamento interno del gruppo, contenendo gli standard di codifica, le procedure di gestione dei ticket su GitHub^G, le convenzioni di denominazione e le metodologie di lavoro adottate.
- **Piano di Progetto^G**: descrive l'allocazione delle risorse, l'analisi dei rischi^G e il preventivo^G economico; viene aggiornato periodicamente con il consuntivo^G delle ore per monitorare lo stato del budget.
- **Piano di Qualifica^G**: specifica la strategia con cui il gruppo garantisce l'assenza di difetti critici, definendo le soglie delle metriche e i test necessari alla qualifica del prodotto.
- **Glossario^G**: raccoglie termini tecnici e acronimi per eliminare le ambiguità terminologiche tra sviluppatori, committenti e utenti finali.
- **Specifica Tecnica**: descrive l'architettura di sistema (modello C4^G), i diagrammi delle classi^G e i design pattern scelti per l'implementazione.
- **Manuale Utente**: guida operativa destinata agli utenti finali per l'onboarding sulla piattaforma Nexum^G e per l'utilizzo degli assistenti AI^G.
- **Lettera di Candidatura e Lettera di Presentazione**: documenti formali per l'ufficializzazione della partecipazione all'appalto e per la consegna dei rilasci RTB^G e PB^G.
- **Verbali interni ed esterni**: tracciano ordine del giorno, discussioni e decisioni prese durante le riunioni di team o con i referenti di *Eggon*.

Le procedure di redazione, verifica e pubblicazione di questi documenti sono quelle definite nel processo di documentazione.

2.1.2 Sviluppo

2.1.2.1 Scopo Il processo di sviluppo deve tradurre i requisiti concordati in un applicativo funzionante e architetturealmente indipendente^G dal sistema Nexum^G, attraversando le fasi di analisi, progettazione, codifica e collaudo con un livello di qualità verificabile in ogni passaggio.

2.1.2.2 Strumenti a supporto

- **Laravel^G 12 e Filament^G**: per la realizzazione rispettivamente del backend applicativo e delle interfacce amministrative.
- **Amazon Bedrock^G**: per l'erogazione delle funzionalità di Intelligenza Artificiale^G.
- **Docker Compose^G**: per l'esecuzione locale dello stack applicativo, nella stessa composizione impiegata per i test di sistema^G.
- **Laravel^G Pint, Larastan e Pest^G PHP^G**: per la formattazione, l'analisi statica^G e i test del codice prodotto.
- **GitHub^G**: per il versionamento del codice, la revisione tra pari^G e l'esecuzione delle pipeline^G di integrazione continua.

2.1.2.3 Attività previste

1. Analisi e tracciamento dei requisiti.
2. Progettazione dell'architettura.
3. Codifica.
4. Collaudo e rilascio.

2.1.2.4 Analisi e tracciamento dei requisiti

1. L'Analista^G separa i flussi di lavoro aziendali di creazione dei contenuti da quelli consulenziali di gestione documentale massiva, in modo che le due aree dell'applicativo restino indipendenti.
2. L'Analista^G classifica gli attori del sistema in utenti operativi^G (primari^G), ovvero gli utenti umani che innescano le operazioni (Amministratore^G, Redattore^G, Analista dati^G, Consulente del lavoro^G), e moduli esterni^G (secondari^G), ovvero le entità di sistema chiamate in causa per completare l'operazione (AI Post Generator^G, AI Analyst^G).
3. L'Analista^G identifica ogni caso d'uso con il codice UC-[Numero] e ne indica le scomposizioni logiche nella forma [Caso Padre] . [Sottocaso^G], ad esempio UC-1.1 per un sottocaso^G di UC-1.
4. L'Analista^G compila per ogni caso d'uso^G una scheda contenente attori coinvolti, trigger^G, precondizioni^G e postcondizioni^G, scenario principale^G in forma di sequenza numerata e scenari alternativi^G.
5. L'Analista^G associa a ogni requisito il riferimento al caso d'uso^G e alla fonte da cui deriva, così che il tracciamento resti verificabile durante la validazione.

2.1.2.5 Progettazione dell'architettura

1. Il Progettista^G definisce un'infrastruttura stand-alone^G che comunichi con i moduli esterni^G di Intelligenza Artificiale^G tramite esecuzione asincrona^G, in modo che le operazioni soggette a latenza^G non blocchino l'interfaccia utente.
2. Il Progettista^G frammenta l'applicativo in macro-componenti a responsabilità singola (ad esempio il modulo di rating^G e feedback, il modulo di split^G documentale e la dashboard^G analitica) e isola le funzionalità trasversali quali raccolta dei log, gestione dei ruoli e telemetria.
3. Il Progettista^G prevede per ogni operazione AI^G a bassa confidenza^G un passaggio di validazione human-in-the-loop^G, ad esempio la revisione dei destinatari estratti o la modifica del testo generato.
4. Il Progettista^G documenta l'architettura nella *Specifica Tecnica* utilizzando il modello C4^G per i livelli di astrazione e i diagrammi delle classi^G UML per la struttura statica.
5. Il Progettista^G sottopone il *design* al Verificatore^G prima che la codifica delle componenti interessate abbia inizio.

2.1.2.6 Codifica

1. Il Programmatore^G prende in carico la propria Issue^G e crea il branch^G di lavoro dedicato prima di scrivere qualsiasi riga di codice.
2. Il Programmatore^G implementa la funzionalità rispettando i pattern architetturali, lo standard stilistico e le convenzioni di denominazione definiti nei sottoparagrafi seguenti.
3. Il Programmatore^G esegue Laravel^G Pint e Larastan in locale e corregge tutte le segnalazioni prima del *commit*.
4. Il Programmatore^G scrive i test previsti dal processo di verifica e ne accerta il superamento in locale.
5. Il Programmatore^G apre la Pull Request^G verso il ramo di integrazione assegnandola al Verificatore^G designato e allegando l'esito dei test.

Pattern architetturali

- **Modularità e riuso:** il codice deve essere suddiviso in moduli con responsabilità singola (ad esempio la libreria dei prompt^G separata dal motore di generazione) e i pattern architetturali devono essere applicati per evitare dipendenze dirette e favorire la manutenibilità.
- **Gestione degli errori e resilienza:** devono essere implementati meccanismi di gestione delle anomalie, *retry* di rete e *fallback* per il caso in cui i servizi AI^G esterni non siano temporaneamente disponibili, coerentemente con gli scenari alternativi^G dei casi d'uso.

- **Tracciabilità e qualità:** ogni modifica deve essere versionata, sottoposta a Code Review^G e coperta da test automatizzati che validino sia le logiche applicative sia la corretta integrazione dei parametri (tono^G, stile^G, metadati^G) passati ai moduli di intelligenza artificiale^G.

Standard stilistici Il codice PHP^G prodotto deve essere conforme allo standard **PSR-12** (PHP^G *Standards Recommendations 12*), adottato ufficialmente dalla comunità PHP^G e seguito nativamente da Laravel^G 12. PSR-12 definisce regole precise su indentazione (4 spazi), apertura delle parentesi, dichiarazione dei tipi e struttura delle classi, garantendo uniformità in tutto il codebase indipendentemente dal membro che scrive il codice.

Convenzioni di denominazione nel codice Le convenzioni seguono gli standard Laravel^G e PSR-12:

- **Classi:** PascalCase — es. `UserController`, `DocumentService`
- **Metodi e funzioni:** camelCase — es. `getDocumentById()`, `processInvoice()`
- **Variabili:** camelCase — es. `$userId`, `$documentPath`
- **Costanti:** UPPER_SNAKE_CASE — es. `MAX_RETRY_COUNT`
- **Tabelle e colonne DB:** snake_case, tabelle al plurale — es. `processed_documents`, `document_type`
- **File Blade:** kebab-case — es. `invoice-list.blade.php`^G
- **Variabili d'ambiente:** UPPER_SNAKE_CASE — es. `MINIO`^G_BUCKET, `DB_CONNECTION`

Strumenti di linting e formattazione

- **Laravel^G Pint:** tool ufficiale di formattazione per Laravel^G, basato su PHP^G CS Fixer. Applica automaticamente PSR-12 al codebase e deve essere eseguito prima di ogni *commit*; il codice non conforme non può essere integrato nel branch^G principale.
- **Larastan:** estensione di PHPStan ottimizzata per Laravel^G. Esegue l'analisi statica^G del codice per rilevare errori di tipo, chiamate a metodi inesistenti e potenziali bug a runtime, senza eseguire il codice. Il livello minimo di analisi adottato è il **livello 5**.

2.1.2.7 Collaudo e rilascio

1. Il Verificatore^G esegue sulla Pull Request^G le attività previste dal processo di verifica e ne registra l'esito nei commenti.
2. Il Verificatore^G misura le metriche di prodotto pertinenti alla modifica e ne riporta i valori nel Piano di Qualifica^G.

3. Il Verificatore^G approva la Pull Request^G solo se tutti i test sono superati e le soglie delle metriche sono rispettate; in caso contrario richiede le correzioni all'autore.
4. Il Responsabile^G autorizza l'integrazione nel ramo di rilascio e la conseguente chiusura della Issue^G collegata.

2.2 Processi di supporto

I processi di supporto comprendono le attività, i metodi e gli strumenti che non producono direttamente il codice, ma che sono indispensabili per sostenere i processi primari. Definirli significa stabilire regole condivise che garantiscono la tracciabilità di ogni decisione tecnica e organizzativa e la coerenza costante di documenti, codice e Issue^G. I processi di supporto applicati dal gruppo sono documentazione, gestione della configurazione, accertamento della qualità, verifica e validazione.

2.2.1 Documentazione

2.2.1.1 Scopo Il processo di documentazione deve garantire che ogni decisione, requisito e scelta tecnica del progetto sia registrata in un artefatto scritto, verificato e accessibile, così che la comunicazione interna resti tracciabile e che stakeholder^G e committente^G possano osservare l'avanzamento del lavoro senza ricorrere a canali informali.

2.2.1.2 Strumenti a supporto

- **LaTeX^G**: per la stesura di tutti i documenti tecnici e formali, con impaginazione uniforme garantita dai preamboli condivisi.
- **GitHub^G**: per l'archiviazione, il tracciamento storico dei file e la revisione asincrona tramite branch^G e Pull Request^G.
- **GitHub^G Projects**: per la gestione del flusso di lavoro tramite la board Kanban "aLittleByte - Dashboard^G".
- **GitHub^G Actions e GitHub^G Pages**: per l'automazione della compilazione e per la pubblicazione del sito web documentale.
- **Python**: per gli script di generazione dinamica della struttura del sito.

2.2.1.3 Attività previste

1. Produzione dei documenti.
2. Verifica e approvazione.
3. Pubblicazione.
4. Manutenzione.

I tre paragrafi che seguono l'elenco delle attività definiscono i vincoli che ogni documento deve rispettare a prescindere dall'attività in corso, ovvero struttura, stato e denominazione; le procedure delle quattro attività sono riportate subito dopo.

2.2.1.4 Struttura dei documenti Tutti i documenti, eccetto i verbali e i diari di bordo, devono presentare la seguente struttura:

- **Copertina:** riporta il logo, l'indirizzo email del gruppo, il titolo del documento, il nome del gruppo e i metadati^G essenziali (redattori, verificatori, destinatari e data).
- **Registro delle modifiche:** tabella posta all'inizio del file per il tracciamento delle versioni. Per ogni aggiornamento indica numero di versione, data, descrizione della modifica, redattore^G, verificatore^G e approvatore.
- **Indice:** elenco gerarchico dei contenuti con rinvii numerici alle pagine, per favorire una navigazione rapida e mirata.
- **Introduzione:** contiene lo scopo del documento, il rinvio al Glossario^G e i riferimenti normativi e informativi.
- **Contenuto:** corpo centrale del testo organizzato in sezioni e sottosezioni numerate (es. 1, 1.1), redatto in stile chiaro ed esecutivo.

Struttura dei verbali

- **Copertina:** riporta gli elementi visivi e identificativi del gruppo, il titolo con l'indicazione del tipo di verbale e della data, i ruoli assunti per la stesura (redattore^G e verificatore^G) e l'elenco dei destinatari.
- **Indice:** elenco cliccabile delle sezioni per facilitare la navigazione del documento.
- **Informazioni generali:** tabella riassuntiva con i dettagli logistici dell'incontro, ovvero luogo o piattaforma utilizzata (Discord^G), orario di inizio e fine ed elenco puntuale dei partecipanti.
- **Ordine del giorno:** elenco sintetico degli argomenti pianificati e discussi durante la riunione.
- **Attività svolte:** resoconto suddiviso in sottoparagrafi che descrive in modo oggettivo le discussioni affrontate, le problematiche analizzate e le decisioni prese per ciascun punto all'ordine del giorno.
- **To-Do List:** sezione conclusiva dedicata al tracciamento delle azioni conseguenti alla riunione. Per ogni *task* devono essere indicati il responsabile^G e il collegamento diretto alla rispettiva Issue^G GitHub^G, garantendo la tracciabilità completa tra decisione e attività operativa.

Struttura dei diari di bordo I diari di bordo sono diapositive realizzate per gli incontri con i docenti sullo stato di avanzamento del lavoro e hanno lo scopo di condividere rapidamente progressi e ostacoli. Devono contenere:

- **Copertina:** logo del team, data della presentazione, numero del gruppo e nomi dei componenti.

- **Cosa abbiamo fatto:** riepilogo sintetico delle attività concluse nell'ultimo periodo.
- **Cosa faremo:** pianificazione^G operativa per i giorni successivi, con i prossimi *task* da completare.
- **Difficoltà e dubbi:** problemi affrontati dal team e dubbi da sottoporre al corpo docente.

2.2.1.5 Stato di un documento Ogni documento attraversa una sequenza definita di stati, che rende immediatamente riconoscibile il grado di affidabilità del contenuto e la figura responsabile del passo successivo:

- **Bozza**^G: il documento è in stesura da parte del redattore^G incaricato. Il contenuto è provvisorio, può essere incompleto e non deve essere considerato attendibile dagli altri membri né citato come riferimento in altri documenti. La relativa Issue^G si trova nella colonna *In Progress* della board Kanban.
- **In verifica:** il redattore^G ritiene la stesura completa e ha aperto la Pull Request^G assegnandola al verificatore^G designato. Il contenuto non può più essere modificato liberamente: ogni intervento deve nascere da un'osservazione registrata nei commenti della Pull Request^G. La relativa Issue^G passa nella colonna *In Review*.
- **Verificato:** il verificatore^G, necessariamente diverso dal redattore^G, ha completato i controlli previsti dal processo di verifica e non rileva ulteriori difetti. Il documento è coerente, corretto e conforme alle presenti Norme di Progetto^G, ma non costituisce ancora un rilascio ufficiale.
- **Approvato:** il Responsabile^G in carica, nel ruolo di approvatore, ha autorizzato il rilascio del documento verso l'esterno. L'approvazione è registrata nel *Registro delle modifiche* nella colonna *Approvatore* e comporta l'incremento del numero di versione maggiore.

Il passaggio da uno stato al successivo non è mai implicito: deve concretizzarsi in un evento tracciabile su GitHub^G (apertura della Pull Request^G, approvazione della revisione, *merge* sul ramo *main*), così che lo stato di ogni documento sia ricostruibile dalla cronologia del repository^G.

2.2.1.6 Denominazione dei documenti Per preservare l'uniformità del repository^G e agevolare la ricerca dei file, la denominazione dei documenti deve rispettare le seguenti convenzioni:

- **Verbali:** adottano la forma <Tipo>_<Data>, dove <Tipo> è *Verbale_Interno* o *Verbale_Esterno* e <Data> è la data della riunione nel formato AAAA_MM_GG. Ad esempio *Verbale_Interno_2026_04_13.pdf* è il verbale interno^G del 13/04/2026.
- **Diari di bordo:** adottano la forma *Diario_di_Bordo_AAAA_MM_GG.pdf*. Ad esempio *Diario_di_Bordo_2026_04_13.pdf* è il diario di bordo^G presentato il 13/04/2026.
- **Altri documenti:** adottano il nome esplicito del tipo di documento, ad esempio Norme di Progetto^G.pdf o Analisi dei Requisiti^G.pdf.

2.2.1.7 Produzione dei documenti

1. Il Responsabile^G stabilisce, in corrispondenza delle scadenze di progetto, quali documenti produrre o aggiornare nell'iterazione corrente.
2. Il Responsabile^G designa per ogni documento un redattore^G e un verificatore^G distinti, verificando l'assenza di conflitti d'interesse, e apre le relative *Issue*^G.
3. Il redattore^G crea un branch^G dedicato al singolo file, inclusi verbali e diari di bordo, così da isolare completamente il lavoro rispetto al ramo principale.
4. Il redattore^G stende il contenuto in LaTeX^G rispettando la struttura, la denominazione e le direttive stilistiche definite nei paragrafi precedenti.
5. Il redattore^G incrementa il numero di versione e inserisce la corrispondente riga nel *Registro delle modifiche* contestualmente alle modifiche apportate.
6. Il redattore^G apre la *Pull Request*^G verso `main` assegnandola al verificatore^G designato, portando così il documento nello stato *In verifica*.

Durante la stesura il redattore^G deve attenersi alle seguenti direttive stilistiche:

- **Struttura dei periodi:** semplificare i periodi più densi, evitando frasi eccessivamente lunghe o ricche di incisi e separando le informazioni con funzioni diverse.
- **Terminologia tecnica:** demandare le definizioni e le spiegazioni dei concetti tecnici al *Glossario*^G, evitando di ripeterle nel corpo del documento.
- **Organizzazione logica:** separare visivamente, con paragrafi distinti o elenchi, le parti descrittive da quelle valutative, dalle motivazioni e dalle conseguenze.
- **Stile prescrittivo:** nelle sezioni che normano un'attività, indicare il ruolo responsabile e l'azione richiesta anziché limitarsi a descrivere la prassi in corso.

2.2.1.8 Verifica e approvazione

1. Il verificatore^G applica al documento le tecniche di analisi statica^G definite nel processo di verifica e annota ogni difetto rilevato nei commenti della *Pull Request*^G.
2. Il verificatore^G controlla che la versione riportata in copertina coincida con l'ultima riga del *Registro delle modifiche* e che i metadati^G siano completi.
3. Il redattore^G applica le correzioni richieste; l'intervento sul testo resta di competenza esclusiva dell'autore, mentre il verificatore^G si limita a segnalare i difetti.
4. Il verificatore^G ripete i passi 1 e 3 finché non rileva più difetti, quindi approva la revisione portando il documento nello stato *Verificato*.
5. Il Responsabile^G, per i documenti destinati all'esterno, registra l'approvazione nel *Registro delle modifiche* incrementando il numero di versione maggiore.

2.2.1.9 Pubblicazione

1. Il verificatore^G che ha approvato la revisione effettua il *merge* della Pull Request^G nel ramo `main`.
2. Il workflow^G di rilascio continuo descritto nel processo di gestione della configurazione compila il sito e pubblica il documento su GitHub^G Pages senza ulteriori interventi manuali.
3. Il redattore^G verifica che il documento risulti effettivamente raggiungibile dal sito del gruppo e, in caso di esito negativo, apre una Issue^G assegnata all'Amministratore^G.

Poiché il *merge* equivale a una pubblicazione, non deve mai essere effettuato su contenuti in stato di bozza^G.

2.2.1.10 Manutenzione

1. Il membro che individua un difetto in un documento già pubblicato apre una Issue^G descrittiva, indicando il documento, la sezione e la natura del difetto.
2. Il Responsabile^G assegna la Issue^G a un redattore^G e a un verificatore^G, secondo la procedura di produzione.
3. Il redattore^G incaricato riporta allo stato di bozza^G la sola parte interessata e le fa ripercorrere l'intero ciclo di verifica e approvazione.

2.2.2 Gestione della configurazione

2.2.2.1 Scopo Il processo di gestione della configurazione deve tracciare, organizzare e controllare l'evoluzione di tutti gli artefatti di progetto, ovvero codice sorgente, documentazione e script, in modo da prevenire conflitti, consentire il recupero di versioni precedenti e garantire che ogni rilascio sia coerente e verificabile.

2.2.2.2 Strumenti a supporto

- **Git^G**: sistema di controllo di versione distribuito con cui viene monitorata ogni variazione ai file sorgente e alla documentazione.
- **GitHub^G**: servizio di hosting del repository^G centralizzato, impiegato anche per la pianificazione^G operativa tramite le Issue^G e per la revisione del lavoro tramite le Pull Request^G.
- **GitHub^G Actions**: per l'esecuzione automatica delle procedure di compilazione, test e rilascio.

2.2.2.3 Attività previste

1. Identificazione della configurazione.
2. Controllo della configurazione.
3. Registrazione dello stato di configurazione.
4. Valutazione della configurazione.

2.2.2.4 Identificazione della configurazione

1. Ogni artefatto di progetto deve risiedere nel repository^G aLittleByte-19/Documentazione e trovare collocazione nell'alberatura secondo la fase di progetto a cui appartiene.
2. Ogni documento deve essere identificato dalla coppia composta dal nome definito nelle convenzioni di denominazione e dal numero di versione riportato in copertina.
3. Ogni componente software deve essere identificata dal branch^G su cui è sviluppata e dalla Issue^G che ne motiva l'esistenza.
4. L'Amministratore^G verifica che non esistano artefatti privi di collocazione o di identificativo e ne dispone la regolarizzazione.

Nomenclatura dei branch e alberatura Il gruppo adotta una convenzione di nomenclatura descrittiva per i branch^G, in modo da rendere immediatamente esplicito l'oggetto del lavoro. La convenzione principale prevede l'uso dello *snake_case* per i documenti generali (es. `piano_di_progetto`, `norme_di_progetto`, `analisi_dei_requisiti`); sono tollerate eccezioni descrittive per i verbali o per attività specifiche di sviluppo (es. `VerbaleEsterno_YYYY_MM_DD` o `develop-sito`).

L'alberatura del repository^G documentale separa le fasi del progetto didattico. Le directory principali sono:

- .github^G: contiene i workflow^G di automazione, gli script Python di rilascio e gli asset condivisi.
- `candidatura`: contiene i documenti relativi alla fase di candidatura.
- RTB^G e PB^G: contengono la documentazione delle rispettive milestone^G, con un'ulteriore categorizzazione interna che separa i verbali in `Verbalì/Verbalì Interni` e `Verbalì/Verbalì Esterni`.

2.2.2.5 Controllo della configurazione

1. Ogni attività, sia essa la stesura di un documento, lo sviluppo di una funzionalità, la correzione di un difetto o il contatto con l'azienda, deve essere preventivamente tracciata da una Issue^G.
2. Il Responsabile^G assegna la Issue^G ai membri incaricati e le applica le etichette che ne descrivono il tipo, ad esempio *documentazione* per la stesura dei documenti.
3. L'assegnatario sviluppa la modifica sul branch^G dedicato: nessun membro è autorizzato a effettuare modifiche dirette sul ramo principale `main`, che è protetto.
4. L'assegnatario apre una Pull Request^G, unico meccanismo ammesso per l'integrazione, assegnando come *reviewer* il verificatore^G designato.
5. L'integrazione è consentita solo a seguito di almeno una revisione positiva da parte di un membro diverso dall'autore; in assenza di approvazione la modifica non entra nella configurazione di progetto.

Automazione e rilascio continuo (CI/CD) Per assicurare che la documentazione fornita agli stakeholder^G sia costantemente aggiornata e pubblica, il gruppo ha implementato un processo di *Continuous Deployment* tramite GitHub^G Actions. Il workflow^G principale, denominato **Site Deploy**, si attiva automaticamente a ogni *push* o *Pull Request*^G verso il ramo **main**, ignorando i file non critici come **README.md** e **.gitignore**, ed esegue nell'ordine i seguenti passi:

1. Predispone l'ambiente virtuale con Python e Node e installa le dipendenze necessarie.
2. Valida i dati del Glossario^G e compila l'applicazione Angular^G che ne consente la consultazione.
3. Esegue lo script `.githubG/site-src/update_index.py`, che genera dinamicamente la struttura statica del sito (HTML, CSS e asset associati) a partire dall'alberatura del repository^G.
4. Esegue i test automatici di navigazione e accessibilità del sito, conservando la diagnostica in caso di esito negativo.
5. Carica e rilascia il risultato sull'ambiente **github-pages**, limitatamente ai *push* sul ramo **main**.

Un esito negativo di uno qualsiasi dei passi impedisce il rilascio: in tal caso l'Amministratore^G deve intervenire con priorità, poiché il malfunzionamento blocca la pubblicazione di tutti i documenti.

2.2.2.6 Registrazione dello stato di configurazione Ogni documento presenta un *Registro delle modifiche* per tenere traccia di tutte le modifiche svolte. Il sistema di versionamento^G adottato è il seguente:

X.Y.Z

Dove:

- **X**: subisce un incremento solo quando il file viene approvato dal membro del gruppo incaricato del ruolo di approvatore;
- **Y**: indica un avanzamento significativo, come l'aggiunta di interi capitoli o moduli software che hanno superato positivamente la fase di verifica;
- **Z**: rappresenta modifiche di dettaglio, correzioni di refusi o aggiornamenti minimi eseguiti durante la stesura o lo sviluppo.

Nell'applicazione di questo schema devono essere rispettate le regole seguenti:

- **Incremento e azzeramento**: all'incremento di una cifra, tutte le cifre alla sua destra vengono riportate a zero. La versione successiva alla 0.4.2, in caso di avanzamento significativo, è quindi la 0.5.0 e non la 0.5.2; la prima versione approvata successiva alla 1.3.1 è la 2.0.0.

- **Versione iniziale:** la prima stesura di un documento nasce con versione 0.1.0. Le versioni con **X** pari a 0 identificano un documento non ancora approvato, quindi soggetto a modifiche sostanziali e non utilizzabile come riferimento verso la proponente^G.
- **Corrispondenza con lo stato del documento:** l'incremento di **Z** accompagna le correzioni apportate in stato di bozza^G o a seguito di osservazioni minori del verificatore^G; l'incremento di **Y** si applica solo a contenuti che hanno superato la verifica; l'incremento di **X** è riservato all'approvatore ed è l'unico che segna il passaggio allo stato *Approvato* definito nella sezione ??.
- **Registrazione obbligatoria:** ogni incremento di versione deve corrispondere a una nuova riga del *Registro delle modifiche*, compilata con versione, data, descrizione sintetica dell'intervento, redattore^G, verificatore^G e approvatore. I campi non pertinenti sono valorizzati con un trattino: nelle righe di sola approvazione sono quindi compilati unicamente versione, data, descrizione e approvatore.
- **Coerenza tra copertina e registro:** la versione riportata in copertina deve sempre coincidere con quella dell'ultima riga del *Registro delle modifiche*, e il controllo di questa corrispondenza è parte delle attività del verificatore^G.
- **Documenti a approccio incrementale**^G: i documenti che crescono insieme al progetto (Norme di Progetto^G, Piano di Progetto^G, Piano di Qualifica^G) sono versionati e verificati sezione per sezione: una nuova versione è ammessa anche in presenza di sezioni ancora vuote, purché quelle già redatte siano state verificate.

2.2.2.7 Valutazione della configurazione

1. Il Verificatore^G controlla, al termine di ogni Sprint^G, che a ciascun requisito dell'Analisi dei Requisiti corrisponda una componente identificata nella configurazione e almeno un test associato.
2. Il Verificatore^G registra nel Piano di Qualifica^G lo stato di copertura dei requisiti risultante dal controllo.
3. Il Responsabile^G pianifica nell'iterazione successiva le attività necessarie a colmare le lacune di copertura rilevate.

2.2.3 Accertamento della qualità

2.2.3.1 Scopo Il processo di accertamento della qualità deve verificare che i processi applicati e i prodotti realizzati siano conformi a quanto stabilito dalle presenti Norme di Progetto^G e dal Piano di Qualifica^G. Per essere privo di conflitti d'interesse, l'accertamento deve essere svolto da un membro diverso da chi ha eseguito l'attività o realizzato la componente sottoposta a controllo.

2.2.3.2 Strumenti a supporto

- **Piano di Qualifica^G**: contiene le soglie delle metriche e il cruscotto di valutazione con l'andamento storico delle misurazioni.
- **GitHub^G Projects**: fornisce i dati di avanzamento e di consuntivo necessari alla misurazione delle metriche di processo.
- **Foglio di rendicontazione oraria**: fornisce i dati di impiego delle risorse per il calcolo delle metriche economiche.

2.2.3.3 Attività previste

1. Accertamento della qualità di processo.
2. Accertamento della qualità di prodotto.

2.2.3.4 Accertamento della qualità di processo

1. L'Amministratore^G misura al termine di ogni Sprint^G le metriche di processo definite nel capitolo sulle metriche e ne registra i valori nel cruscotto del Piano di Qualifica^G.
2. L'Amministratore^G confronta ogni valore con la soglia accettabile e segnala al gruppo le metriche fuori soglia durante la Sprint^G Retrospective.
3. Il gruppo tratta ogni segnalazione secondo la procedura del processo di miglioramento dei processi.

2.2.3.5 Accertamento della qualità di prodotto

1. Il Verificatore^G misura le metriche di prodotto sulle componenti sviluppate nell'iterazione, avvalendosi degli esiti dei test e degli strumenti di analisi statica^G.
2. Il Verificatore^G registra i valori rilevati nel Piano di Qualifica^G e non approva le modifiche che portino una metrica sotto la soglia accettabile.
3. Il Verificatore^G verifica che ogni requisito dichiarato soddisfatto sia coperto da almeno un test superato, riportando l'esito nel tracciamento dei requisiti.

2.2.4 Verifica

2.2.4.1 Scopo Il processo di verifica deve garantire l'assenza di difetti e l'aderenza di ogni componente prodotta, documentale o software, alle specifiche e ai vincoli stabiliti, rispondendo alla domanda *“Did I build the system right?”*. L'intera reportistica dei controlli effettuati, dei parametri misurati e degli esiti dei collaudi è raccolta nel Piano di Qualifica^G.

2.2.4.2 Strumenti a supporto

- **Pull Request^G di GitHub^G**: sede in cui vengono registrati i difetti rilevati e le relative correzioni.
- **Larastan e Laravel^G Pint**: per l'analisi statica^G automatica e la verifica di conformità stilistica del codice.
- **Pest^G PHP^G**: per la scrittura e l'esecuzione dei test di unità e di integrazione.
- **Docker Compose^G**: per l'esecuzione dei test di sistema^G sull'ambiente completo.

2.2.4.3 Attività previste

1. Analisi statica^G.
2. Analisi dinamica^G.

La verifica interviene al termine di ogni fase di lavoro, con lo scopo di accertare che il risultato prodotto sia coerente con gli ingressi da cui è stato derivato e che non siano stati introdotti difetti.

2.2.4.4 Analisi statica L'analisi statica^G è una tecnica di verifica applicata all'oggetto di studio senza richiederne l'esecuzione, e consente di individuare difetti nelle prime fasi del ciclo di vita. La procedura è la seguente:

1. Il Verificatore^G sceglie la tecnica di lettura da applicare tra walkthrough^G e *inspection*, in funzione dell'estensione dell'oggetto e del tempo disponibile.
2. Il Verificatore^G esegue la lettura e annota ogni difetto rilevato nei commenti della Pull Request^G, indicando il punto preciso a cui si riferisce.
3. Il Verificatore^G discute i difetti con l'autore in modo costruttivo; l'applicazione materiale delle correzioni resta di competenza dell'autore originale, a garanzia di coerenza e responsabilità.
4. Il Verificatore^G non approva la Pull Request^G finché tutti i difetti rilevati non risultano risolti.

Le due tecniche di lettura adottate sono:

- **Walkthrough^G**: revisione estensiva e senza preconcetti, in cui il verificatore^G analizza il prodotto passo dopo passo simulandone mentalmente la logica e verificando il rispetto delle regole prestabilite. È molto approfondita ma dispendiosa in termini di tempo e richiede un intervento interamente manuale.
- **Inspection**: controllo mirato e guidato dall'esperienza, in cui il verificatore^G utilizza *checklist* specifiche per le criticità più comuni. È meno estesa del walkthrough^G, ma più rapida e ripetibile.

Analisi statica automatica In affiancamento alle tecniche manuali, il team adotta **Larastan** (PHPStan per Laravel^G) come strumento di analisi statica^G automatica del codice. Larastan viene eseguito al livello di analisi **5** e rileva automaticamente i *code smell* tracciati dalla metrica **MPD12**, ad esempio metodi troppo complessi, variabili non tipizzate e chiamate a metodi inesistenti. L'esecuzione avviene in locale prima di ogni *commit* e nella pipeline di *Continuous Integration*: il codice che non supera l'analisi non può essere integrato nel branch^G principale.

2.2.4.5 Analisi dinamica L'analisi dinamica^G presuppone l'esecuzione dell'oggetto sottoposto a verifica ed esamina le reazioni di specifiche porzioni del software applicandovi un numero definito di casi prova, che devono possedere i requisiti di ripetibilità e automazione. Lo stato di ogni test è indicato con una delle seguenti sigle:

- **NI**: non implementato
- **I**: implementato
- **S**: superato
- **NS**: non superato

La procedura che il Programmatore^G deve seguire prima di aprire una Pull Request^G è la seguente:

1. Scrivere i test di unità^G e i test di integrazione^G che coprono la logica del codice prodotto.
2. Verificare che tutti i test passino in locale con il comando `./vendor/bin/pestG`.
3. Calcolare la *code coverage* con `./vendor/bin/pestG --coverage`, che richiede il driver **Xdebug** attivo nel container^G PHP-FPM^G. La copertura minima richiesta è dell'**80%**, in conformità con la metrica **MPC-CC**.
4. Allegare l'esito del report nella descrizione della Pull Request^G.

La pipeline di *Continuous Integration* verifica automaticamente questi requisiti a ogni *push*: una Pull Request^G con copertura inferiore alla soglia o con test falliti non può essere approvata.

Test di unità I test di unità^G verificano in isolamento singole componenti software, quali funzioni, classi o metodi, separandole dal resto dell'architettura, e confermano che ogni unità generi i risultati previsti a partire da input determinati. Il framework adottato è **Pest**^G **PHP**^G, incluso nativamente in Laravel^G 12, e i test risiedono nella directory `tests/Unit/`.

Test di integrazione I test di integrazione^G confermano che le componenti del software operino tra loro in modo sinergico e privo di errori. Sono scritti con **Pest**^G **PHP**^G, risiedono in `tests/Feature/` ed eseguono richieste HTTP reali contro l'applicazione Laravel^G utilizzando il database PostgreSQL^G di test definito in `.env.testing`, verificando l'interazione tra controller, service layer e database. Anche per questi test vale il requisito di copertura minima dell'**80%**.

Test di sistema I test di sistema^G accertano che il comportamento globale del prodotto rispetti i requisiti stabiliti nell'Analisi dei Requisiti^G. Vengono eseguiti sull'applicazione deployata nell'ambiente Docker Compose^G completo (Nginx^G, PHP-FPM^G, PostgreSQL^G, Redis^G, MinIO^G), simulando scenari end-to-end che attraversano tutti i livelli dello stack; MinIO^G è utilizzato come object storage compatibile con le API^G S3^G. Ogni test di sistema^G deve essere tracciato ai requisiti funzionali^G e, ove applicabile, al relativo caso d'uso^G, tramite i codici di riferimento riportati nel Piano di Qualifica^G.

Test di regressione I test di regressione^G accertano che gli interventi correttivi o evolutivi non abbiano introdotto difetti nelle funzionalità già esistenti. Non richiedono la scrittura di test dedicati: sono garantiti dall'esecuzione automatica dell'intera suite Pest^G PHP^G nella pipeline di *Continuous Integration* a ogni *push* o apertura di *Pull Request*^G. Un test precedentemente superato che risulti fallito segnala una regressione e blocca l'integrazione fino alla risoluzione.

2.2.5 Validazione

2.2.5.1 Scopo Il processo di validazione deve confermare che le caratteristiche del prodotto siano conformi ai bisogni d'uso e agli obiettivi della proponente^G, rispondendo alla domanda “*Did I build the right system?*”. Mentre la verifica analizza la correttezza tecnica dell'esecuzione, la validazione si sposta sul piano dell'efficacia, accertando che le funzionalità sviluppate risolvano effettivamente le problematiche operative individuate.

2.2.5.2 Strumenti a supporto

- **Analisi dei Requisiti^G**: fornisce i requisiti classificati e il loro tracciamento verso i casi d'uso.
- **Piano di Qualifica^G**: raccoglie i test di accettazione^G e i relativi esiti.
- **Google Meet e posta elettronica**: per la raccolta strutturata dei *feedback* della proponente^G.

2.2.5.3 Attività previste

1. Tracciamento dei requisiti.
2. Esecuzione dei test di accettazione^G.

2.2.5.4 Tracciamento dei requisiti

1. L'Analista^G raccoglie e formalizza nell'Analisi dei Requisiti^G tutte le necessità espresse da *Eggon*, classificandole per tipologia e urgenza.
2. L'Analista^G associa a ogni requisito i casi d'uso^G che lo realizzano, così che il perimetro del prodotto resti verificabile.
3. Il Verificatore^G associa a ogni requisito i test che lo coprono e ne aggiorna lo stato nel Piano di Qualifica^G al termine di ogni Sprint^G.

4. Un requisito può essere dichiarato soddisfatto solo quando il codice che lo implementa è coperto da test superati.

2.2.5.5 Esecuzione dei test di accettazione

1. Il Verificatore^G predispone i test di accettazione^G a partire dagli scenari dei casi d'uso^G concordati con la proponente^G.
2. Il Responsabile^G presenta a *Eggon* le funzionalità completate durante gli incontri esterni periodici e ne raccoglie i *feedback*.
3. Il redattore^G del verbale esterno^G registra i rilievi espressi dall'azienda e li converte in Issue^G assegnate.
4. Il Verificatore^G riporta nel Piano di Qualifica^G l'esito di ogni test di accettazione^G, utilizzando gli stati definiti per l'analisi dinamica^G.

2.3 Processi organizzativi

I processi organizzativi governano il modo in cui il gruppo lavora e si affiancano ai processi primari e di supporto per tutta la durata del progetto. In conformità con lo standard ISO/IEC 12207, il gruppo *aLittleByte* applica i processi di gestione dei processi, infrastruttura, formazione e miglioramento dei processi.

2.3.1 Gestione dei processi

2.3.1.1 Scopo Il processo di gestione dei processi deve assicurare che gli obiettivi di progetto siano raggiunti nel rispetto dei vincoli di costo, delle tempistiche stimate e dei requisiti concordati con la proponente^G, attraverso la pianificazione^G delle attività, il monitoraggio dell'avanzamento, la prevenzione dei rischi e l'assegnazione trasparente di ruoli e responsabilità.

2.3.1.2 Strumenti a supporto

- **GitHub^G Issue^G**: per la classificazione delle attività da svolgere e la loro assegnazione ai membri del gruppo.
- **GitHub^G Projects**: per la gestione del *backlog* e del flusso Kanban tramite la board "aLittleByte - Dashboard^G".
- **Google Fogli**: per la rendicontazione delle ore di lavoro di ciascun membro e di ciascun ruolo.
- **Piano di Progetto^G**: per la registrazione di preventivi^G, consuntivi^G e analisi dei rischi^G.
- **Discord^G**: per lo svolgimento delle riunioni interne da remoto.

2.3.1.3 Attività previste

1. Pianificazione^G dell'iterazione.
2. Esecuzione e controllo.
3. Revisione e valutazione.
4. Conclusione dell'iterazione.

Il gruppo adotta un approccio iterativo^G, suddividendo il ciclo di vita del progetto in Sprint^G di durata fissa di due settimane. Le quattro attività elencate scandiscono ogni iterazione; i paragrafi successivi alle relative procedure normano gli elementi trasversali al processo, ovvero ruoli, coordinamento e tracciamento.

2.3.1.4 Pianificazione dell'iterazione

1. All'avvio di ogni Sprint^G il gruppo si riunisce e assegna i ruoli che ciascun membro ricoprirà per l'intera durata dell'iterazione, ruotandoli rispetto all'iterazione precedente.
2. Il Responsabile^G seleziona dal *backlog* gli obiettivi operativi dell'iterazione e li scompone in *task* eseguibili singolarmente.
3. Il Responsabile^G apre una *Issue*^G per ogni *task*, la assegna a un membro bilanciando i carichi di lavoro e la colloca nella colonna *Todo* della board.
4. Il Responsabile^G designa per ogni *task* il verificatore^G che ne curerà la revisione, accertandosi che non coincida con l'assegnatario.
5. Il Responsabile^G compila nel Piano di Progetto^G il preventivo^G di periodo, indicando le ore previste per ruolo e i rischi attesi.

2.3.1.5 Esecuzione e controllo

1. L'assegnatario sposta la propria *Issue*^G nella colonna *In Progress* nel momento in cui inizia a lavorarci.
2. L'assegnatario svolge il *task* seguendo la procedura del processo pertinente e, in caso di difficoltà, chiede chiarimenti nella sezione commenti della *Issue*^G.
3. L'assegnatario apre la *Pull Request*^G al termine del lavoro e sposta la *Issue*^G nella colonna *In Review*.
4. Il Responsabile^G monitora l'avanzamento della board e ridistribuisce i *task* qualora rilevi il rischio di superare le scadenze fissate.

Gestione del flusso di lavoro tramite Kanban Per garantire un controllo visivo e immediato sullo stato di avanzamento delle *Issue*^G, il gruppo gestisce i *task* tramite la board GitHub^G Projects “aLittleByte - Dashboard^G”, modellata secondo i principi Kanban e articolata nelle seguenti colonne:

- **Todo:** *task* selezionati e pronti per essere affrontati nel ciclo di lavoro corrente.
- **In Progress:** attività su cui il team sta lavorando attivamente, con le *Issue*^G formalmente assegnate tramite il campo *Assignee*.
- **In Review:** *task* completati e in attesa di *Peer Review* tramite l'approvazione della relativa *Pull Request*^G.
- **Done:** lavoro verificato, approvato e integrato definitivamente nel ramo principale.

2.3.1.6 Revisione e valutazione

1. Al termine dell'iterazione il gruppo svolge la Sprint^G Review, verificando i risultati prodotti e discutendo le criticità tecniche emerse.
2. Il Responsabile^G confronta le ore preventivate con quelle consuntivate e motiva nel Piano di Progetto^G gli scostamenti principali, quali sottostime, attività impreviste e dipendenze esterne.
3. Il Responsabile^G aggiorna le previsioni di costo a finire sulla base degli scostamenti rilevati.
4. Il gruppo svolge la Sprint^G Retrospective, individuando i punti di forza da mantenere e le inefficienze organizzative da correggere.
5. Il Responsabile^G revisiona il Registro dei Rischi documentando, per ogni criticità emersa o rischio^G inatteso manifestatosi, la causa scatenante, l'impatto effettivo e l'azione di mitigazione^G prevista, che deve essere assegnata a un membro.

2.3.1.7 Conclusione dell'iterazione Il gruppo adotta la seguente *Definition of Done*: un'attività è conclusa quando la relativa *Pull Request*^G è stata approvata da un verificatore^G diverso dall'assegnatario e integrata nel ramo principale. La procedura di chiusura è la seguente:

1. L'assegnatario registra nel foglio di rendicontazione le ore effettivamente impiegate sul *task*.
2. Il verificatore^G che ha approvato la *Pull Request*^G registra a propria volta le ore impiegate nell'attività di verifica.
3. L'assegnatario chiude la *Issue*^G spostandola nella colonna *Done* ed elimina il branch^G di lavoro.
4. Il Responsabile^G compila nel Piano di Progetto^G il consuntivo^G di periodo e riporta nel *backlog* i *task* non completati.

2.3.1.8 Ruoli Durante una medesima iterazione un membro può assumere più ruoli, a condizione che ciò non generi conflitti d'interesse: in particolare, nessuno può verificare un artefatto che ha redatto o sviluppato. I ruoli previsti sono riportati di seguito in ordine alfabetico.

Amministratore L'Amministratore^G è la figura di supporto metodologico e tecnologico del team e deve garantire che il gruppo possa procedere con le attività senza impedimenti di natura tecnica o organizzativa. Nello specifico deve:

- gestire gli ambienti di lavoro e mantenere operativa l'infrastruttura descritta nel processo di infrastruttura;
- supervisionare e aggiornare periodicamente i software e i canali adottati per la comunicazione e la collaborazione asincrona;
- garantire la corretta applicazione delle procedure di versionamento^G a tutti i file e i documenti prodotti;
- curare la stesura, la revisione e l'aggiornamento delle presenti Norme di Progetto^G, mantenendole allineate alle reali necessità del gruppo;
- misurare al termine di ogni Sprint^G le metriche di processo e riportarle nel Piano di Qualifica^G.

Analista L'Analista^G è il punto di incontro tra le esigenze della proponente^G e il lavoro tecnico del team, e deve trasformare le richieste in requisiti chiari, completi e privi di ambiguità. Nello specifico deve:

- redigere e mantenere aggiornata l'Analisi dei Requisiti^G, catalogando casi d'uso e specifiche del progetto;
- interfacciarsi con la proponente^G per risolvere incertezze, chiedere chiarimenti e concordare variazioni ai requisiti;
- collaborare con progettisti^G e programmatici^G affinché le specifiche siano interpretate correttamente e implementate senza deviazioni.

Programmatore Il Programmatore^G deve tradurre l'architettura e le specifiche definite dal Progettista^G in codice funzionante, pulito ed efficiente. Nello specifico deve:

- scrivere il codice sorgente rispettando le norme stilistiche e gli standard di qualità definiti nel processo di sviluppo;
- sviluppare ed eseguire i test previsti dal processo di verifica prima di sottoporre il proprio lavoro a revisione;
- collaborare con il resto del team per individuare i difetti, correggere i *bug* e implementare le nuove funzionalità in modo coerente con il resto del sistema.

Progettista Il Progettista^G deve ideare l'architettura del sistema in modo che soddisfi i requisiti funzionali e di qualità concordati con la proponente^G. Nello specifico deve:

- valutare e decidere linguaggi, *framework* e strumenti tecnici più idonei alla realizzazione del progetto;
- definire la struttura del sistema individuando i macro-componenti e le modalità di comunicazione tra essi;
- supervisionare la stesura del codice, accertando che l'implementazione rispetti le specifiche tecniche;
- redigere e mantenere aggiornati, a partire dalla fase PB^G, la *Specifica Tecnica* e il *Manuale Utente*.

Responsabile Il Responsabile^G agisce come *Project Manager* del gruppo e deve coordinare il team, pianificare le iterazioni, ottimizzare l'uso delle risorse e rappresentare il gruppo verso la proponente^G e il corpo docente. Nello specifico deve:

- distribuire e assegnare i compiti tramite le *Issue*^G, secondo la procedura di pianificazione^G dell'iterazione;
- monitorare l'avanzamento del lavoro e intervenire quando le misurazioni evidenzino scostamenti dal Piano di Progetto^G;
- approvare i documenti destinati all'esterno, registrando l'approvazione nel *Registro delle modifiche*;
- redigere e mantenere aggiornati i verbali interni ed esterni, il Diario di Bordo^G, il Piano di Progetto^G e il Piano di Qualifica^G.

Verificatore Il Verificatore^G è il garante della qualità e deve accertare che ogni elemento prodotto, documentale o software, rispetti i requisiti stabiliti e gli standard prefissati. Nello specifico deve:

- revisionare tramite *Pull Request*^G i documenti redatti dagli altri membri, accertandone correttezza, completezza e assenza di ambiguità;
- pianificare e condurre i test previsti dal processo di verifica sul software prodotto;
- registrare gli esiti delle verifiche e segnalare tempestivamente difetti e non conformità agli autori;
- individuare i punti critici del Way of Working^G e proporre iniziative di miglioramento.

2.3.1.9 Coordinamento Il coordinamento avviene tramite riunioni pianificate e canali di comunicazione differenziati per formalità e urgenza.

Riunioni interne

- **Incontro settimanale:** il gruppo si riunisce una volta alla settimana, indicativamente il lunedì, per fare il punto della situazione, organizzare i *task* dei giorni successivi e individuare soluzioni condivise agli ostacoli emersi.
- **Riunioni straordinarie:** in presenza di urgenze, *bug* bloccanti o decisioni tecniche complesse, i membri interessati possono convocare incontri aggiuntivi.
- **Piattaforma:** le riunioni si svolgono prevalentemente online sui canali vocali di Discord^G.
- **Verbalizzazione:** al termine di ogni incontro deve essere redatto un Verbale Interno^G secondo la struttura definita nella sezione ??.

Riunioni esterne Gli incontri con la proponente^G si tengono in via telematica su **Google Meet** con cadenza bisettimanale. A conclusione di ogni incontro il gruppo redige un Verbale Esterno^G che riepiloga le tematiche affrontate, le decisioni prese e le azioni concordate; il documento viene poi inoltrato al referente aziendale per il controllo incrociato e, previa conferma, sottoposto alla sua firma per approvazione definitiva.

Comunicazioni interne

- **WhatsApp:** per le comunicazioni rapide e il coordinamento tra i membri del gruppo.
- **Discussioni su GitHub**^G: per le discussioni all'interno delle *Issue*^G e delle *Pull Request*^G, in modo da mantenere tracciate le decisioni e coinvolgere solo i membri interessati.

Comunicazioni esterne

- **E-mail:** canale ufficiale per le comunicazioni formali, l'invio della documentazione e gli aggiornamenti sullo stato di avanzamento; tutte le comunicazioni sono gestite tramite l'indirizzo `alittlebyte.swe@gmail.com`.
- **Telegram:** per scambi rapidi, chiarimenti immediati e coordinamento logistico con il referente aziendale.

Le comunicazioni esterne sono di competenza del Responsabile^G, che è l'unico autorizzato a impegnare il gruppo verso la proponente^G.

2.3.1.10 Tracciamento delle ore Il gruppo adotta un sistema di rendicontazione oraria basato su un foglio di calcolo condiviso su **Google Fogli**, accessibile a tutti i membri e organizzato per membro e per ruolo ricoperto. La procedura è la seguente:

1. Per ciascun *task* svolto il membro indica le ore previste e le ore effettivamente impiegate, consentendo il calcolo automatico degli scostamenti e del saldo ore rimanente.

2. Ogni membro aggiorna il proprio registro orario entro la fine di ogni settimana.
3. Il Responsabile^G revisiona il bilancio orario al termine di ogni Sprint^G, verificando che i dati siano completi e coerenti con le attività svolte.
4. Il Responsabile^G controlla che nessun membro abbia esaurito prematuramente le ore assegnate a un determinato ruolo e, in caso di criticità, ridistribuisce i *task* nelle iterazioni successive.

Le ore sono catalogate secondo due dimensioni: **per ruolo**, tra Responsabile^G, Amministratore^G, Analista^G, Progettista^G, Programmatore^G e Verificatore^G; **per membro**, in modo che sia sempre noto il monte ore residuo individuale per ciascuna figura professionale. Le ore dedicate alla formazione sono registrate a carico del ruolo per cui la formazione è funzionale.

2.3.1.11 Tracciamento delle azioni

1. Al termine di ogni riunione il membro designato redige il verbale secondo la struttura definita nella sezione ??.
2. Il redattore^G del verbale converte ogni compito emerso in una Issue^G su GitHub^G, assegnandola ai membri incaricati.
3. Il redattore^G del verbale riporta nella To-Do List il collegamento diretto a ciascuna Issue^G creata, in modo che ogni decisione risulti collegata all'attività che la attua.
4. L'assegnatario che riscontri difficoltà nello svolgimento del compito utilizza la sezione commenti della Issue^G per chiedere chiarimenti al resto del team.

2.3.2 Infrastruttura

2.3.2.1 Scopo Il processo di infrastruttura deve istituire e mantenere operativo l'insieme degli strumenti che sostengono tutti gli altri processi di progetto. Il gruppo concentra l'infrastruttura su un numero ridotto di strumenti, privilegiando quelli integrati nativamente in GitHub^G, in modo da limitare la dispersione delle informazioni e ridurre il costo di apprendimento.

2.3.2.2 Strumenti a supporto

- GitHub^G e Git^G: per il versionamento, la collaborazione e l'automazione.
- GitHub^G Actions e GitHub^G Pages: per l'esecuzione delle automazioni e la pubblicazione del sito documentale.
- Docker Compose^G: per la replicabilità dell'ambiente di sviluppo del prodotto.

2.3.2.3 Attività previste

1. Implementazione dell'infrastruttura.
2. Creazione dell'infrastruttura.
3. Manutenzione dell'infrastruttura.

2.3.2.4 Implementazione dell'infrastruttura L'infrastruttura del gruppo è costituita dalle quattro componenti descritte nei sottoparagrafi seguenti, ciascuna a servizio di un insieme definito di processi.

Infrastruttura di versionamento e collaborazione L'intera documentazione di progetto è ospitata nel repository^G aLittleByte-19/Documentazione dell'organizzazione GitHub^G del gruppo. Gli elementi principali sono:

- **Ramo main:** contiene esclusivamente contenuti verificati e rappresenta la fonte di verità del progetto. È protetto: nessun membro può effettuare *push* diretti e ogni modifica deve transitare da una Pull Request^G approvata.
- **Ramo develop:** ramo di integrazione del codice del prodotto, su cui confluiscono i rami di lavoro dei singoli programmatori^G prima del consolidamento. L'allineamento periodico di questo ramo è un'attività pianificata e tracciata come qualsiasi altro *task*.
- **Issue^G e Pull Request^G:** costituiscono rispettivamente l'*issue tracking system* e lo strumento di revisione del gruppo, secondo le modalità definite nel processo di gestione della configurazione.
- **GitHub^G Projects:** la board “aLittleByte - Dashboard^G” raccoglie le Issue^G di progetto e ne governa il flusso Kanban.

Infrastruttura documentale

- **LaTeX^G:** tutti i documenti sono sorgenti `.tex` versionati insieme al resto del progetto. I loghi e gli asset grafici condivisi risiedono in `.githubG/site-src/assets`, così che una modifica all'identità visiva si propaghi automaticamente a tutti i documenti alla successiva compilazione.
- **Python:** lo script `.githubG/site-src/update_index.py` genera la struttura statica del sito documentale a partire dall'alberatura del repository^G, evitando la manutenzione manuale delle pagine HTML; la generazione delle anteprime dei PDF si appoggia all'utilità `poppler-utils`.
- **Applicazione del Glossario^G:** il Glossario^G è pubblicato anche come applicazione Angular^G contenuta in `.githubG/glossary-app`, che ne consente la consultazione e la ricerca dal browser. I dati risiedono nel file `glossary.json` e sono sottoposti a validazione automatica prima di ogni rilascio, per impedire la pubblicazione di voci malformate o di ancora non risolvibili dai collegamenti presenti nei documenti.
- **GitHub^G Pages:** ospita il sito del gruppo, che rende disponibile agli stakeholder^G l'ultima versione integrata di ogni documento.

Infrastruttura di sviluppo

- **Ambiente di esecuzione:** l'applicativo viene eseguito localmente tramite Docker Compose^G con i servizi Nginx^G, PHP-FPM^G, PostgreSQL^G, Redis^G e MinIO^G, nella stessa composizione utilizzata per i test di sistema^G.
- **Framework applicativi:** il backend è realizzato con Laravel^G 12 e le interfacce amministrative con Filament^G; le funzionalità di Intelligenza Artificiale^G sono erogate tramite Amazon Bedrock^G.
- **Strumenti di qualità:** Laravel^G Pint, Larastan e Pest^G PHP^G, secondo le soglie e le procedure fissate nel processo di verifica.

Infrastruttura di comunicazione Gli strumenti adottati sono **Discord**^G per le riunioni interne, **WhatsApp** per il coordinamento rapido, **Google Meet** e la casella `alittlebyte.swe@gmail.com` per i rapporti con la proponente^G, **Telegram** per i chiarimenti immediati con il referente aziendale e **Google Fogli** per la rendicontazione oraria. Le finalità e i vincoli d'uso di ciascun canale sono definiti nel paragrafo sul coordinamento.

2.3.2.5 Creazione dell'infrastruttura

1. Chi rileva la necessità di un nuovo strumento o di una nuova automazione apre una Issue^G che ne descrive la finalità e i processi che ne beneficerebbero.
2. Il gruppo delibera l'adozione dello strumento in riunione interna e la decisione è registrata nel relativo verbale interno^G.
3. L'Amministratore^G configura lo strumento e ne documenta l'uso all'interno delle presenti Norme di Progetto^G prima che il gruppo inizi a utilizzarlo.
4. L'Amministratore^G integra la modifica tramite Pull Request^G verificata, come previsto per ogni altro artefatto di progetto.

2.3.2.6 Manutenzione dell'infrastruttura

1. L'Amministratore^G verifica periodicamente il corretto funzionamento delle automazioni e degli strumenti adottati.
2. Chi riscontra un malfunzionamento apre una Issue^G assegnata all'Amministratore^G, descrivendo l'effetto del guasto sulle attività in corso.
3. L'Amministratore^G interviene con priorità sui malfunzionamenti che bloccano il lavoro degli altri membri, in particolare su quelli che impediscono il rilascio della documentazione.
4. Le richieste di adeguamento emerse durante la Sprint^G Retrospective sono trattate come le altre iniziative di miglioramento definite nel processo di miglioramento dei processi.

2.3.3 Formazione

2.3.3.1 Scopo Il processo di formazione deve garantire che ogni membro acquisisca le competenze necessarie a ricoprire i ruoli previsti dal Way of Working^G, evitando che la conoscenza di uno strumento resti concentrata in una sola persona e diventi un rischio^G per l'avanzamento del progetto.

2.3.3.2 Strumenti a supporto

- **Documentazione ufficiale delle tecnologie adottate:** fonte primaria di studio, integrata da risorse divulgative solo quando risulti carente sul piano pratico.
- **Riunioni interne su Discord**^G: sede in cui le conoscenze acquisite vengono condivise con il resto del gruppo.
- **Foglio di rendicontazione oraria:** strumento con cui le ore di formazione vengono registrate e rese visibili nel bilancio dello Sprint^G.

2.3.3.3 Attività previste

1. Individuazione delle competenze necessarie.
2. Raccolta del materiale di formazione.
3. Attuazione della formazione.

2.3.3.4 Individuazione delle competenze necessarie A partire dai requisiti raccolti nell'Analisi dei Requisiti^G e dalle scelte tecnologiche formalizzate dal Progettista^G, il gruppo ha individuato i seguenti ambiti di formazione:

- **Documentazione e automazioni:** LaTeX^G per la stesura dei documenti, Python per gli script di generazione del sito, Angular^G per l'applicazione del Glossario^G, Git^G e GitHub^G (Issue^G, Pull Request^G, Projects, Actions, Pages) per il versionamento e le automazioni.
- **Progettazione:** UML per i diagrammi delle classi^G e dei casi d'uso, modello C4^G per la descrizione dell'architettura su più livelli di astrazione.
- **Sviluppo:** PHP^G con lo standard PSR-12, Laravel^G 12 e Filament^G per l'applicativo, PostgreSQL^G per la persistenza, Redis^G per code e cache, MinIO^G come object storage e Docker Compose^G per l'orchestrazione dell'ambiente locale.
- **Verifica:** Pest^G PHP^G per i test di unità e di integrazione, Larastan e Laravel^G Pint per l'analisi statica^G e la formattazione.
- **Moduli di Intelligenza Artificiale:** Amazon Bedrock^G e i concetti relativi ai Large Language Model^G, alla costruzione dei prompt^G, all'OCR^G e all'Entity Resolution^G, indispensabili per lo sviluppo dell'AI Assistant Generativo^G e dell'AI Co-Pilot^G.

2.3.3.5 Raccolta del materiale di formazione I riferimenti condivisi tra i membri del gruppo sono i seguenti:

- **Laravel^G** e **Filament^G**: <https://laravel.com/docs> e <https://filamentphp.com/docs>
- **PHP^G** e **PSR-12**: <https://www.php.net/docs.php> e <https://www.php-fig.org/psr/psr-12/>
- **Pest^G** **PHP^G**: <https://pestphp.com/docs/installation>
- **PostgreSQL^G** e **Redis^G**: <https://www.postgresql.org/docs/> e <https://redis.io/docs/latest/>
- **Docker Compose^G** e **MinIO^G**: <https://docs.docker.com/compose/> e <https://min.io/docs/minio/linux/index.html>
- **Amazon Bedrock^G**: <https://docs.aws.amazon.com/bedrock/>
- **Git^G** e **GitHub^G**: <https://git-scm.com/doc> e <https://docs.github.com/en>
- **LaTeX^G** e **Angular^G**: <https://www.latex-project.org/help/documentation/> e <https://angular.dev/overview>
- **Modello C4^G** e **UML**: <https://c4model.com/> e <https://www.uml-diagrams.org/>

Chi individua una risorsa più efficace di quelle elencate è tenuto a segnalare al gruppo, così che l'elenco resti aggiornato.

2.3.3.6 Attuazione della formazione La formazione avviene principalmente in modo asincrono e individuale, secondo la procedura seguente:

1. Prima di assumere un ruolo nuovo in un'iterazione, il membro interessato richiede al **Responsabile^G** di riservare parte del proprio carico a un'attività di formazione, che viene pianificata e tracciata come una normale *Issue^G*.
2. Il membro registra le ore dedicate allo studio a carico del ruolo per cui la formazione è funzionale, secondo la procedura di tracciamento delle ore.
3. Il membro che affronta per primo una tecnologia condivide con il gruppo, nella riunione interna successiva, i riferimenti utili e le difficoltà incontrate, così da abbattere il tempo di apprendimento per gli altri componenti.
4. Il gruppo riesamina il fabbisogno formativo durante la **Sprint^G** Retrospective: se un ritardo o un difetto è attribuibile a una competenza insufficiente, viene pianificata un'attività di formazione mirata nell'iterazione successiva anziché la sola riassegnazione del *task*.

2.3.4 Miglioramento dei processi

2.3.4.1 Scopo Il processo di miglioramento deve consentire al gruppo di aumentare l'efficacia e l'efficienza del proprio Way of Working^G a ogni iterazione, traducendo le misurazioni raccolte in modifiche concrete alle procedure normate nel presente documento.

2.3.4.2 Strumenti a supporto

- **Piano di Qualifica^G**: contiene il cruscotto di valutazione con l'andamento delle metriche e la registrazione delle iniziative di miglioramento.
- **Sprint^G Retrospective**: sede in cui il gruppo analizza le inefficienze e delibera le azioni correttive.
- **Presenti Norme di Progetto^G**: artefatto in cui ogni miglioramento deliberato deve essere formalizzato come regola operativa.

2.3.4.3 Attività previste

1. Valutazione dei processi.
2. Miglioramento dei processi.

2.3.4.4 Valutazione dei processi

1. L'Amministratore^G presenta al gruppo, in occasione della Sprint^G Retrospective, le misurazioni raccolte nel cruscotto di valutazione del Piano di Qualifica^G.
2. Il gruppo individua le metriche che si collocano sotto la soglia di accettabilità e le inefficienze operative percepite ma non ancora misurate.
3. Il gruppo risale alla causa di ciascuna criticità, distinguendo i difetti di procedura da quelli di competenza o di strumento.

2.3.4.5 Miglioramento dei processi

1. Il gruppo elabora per ogni criticità una specifica iniziativa di miglioramento e la documenta nel Piano di Qualifica^G.
2. L'Amministratore^G recepisce l'iniziativa nelle presenti Norme di Progetto^G sotto forma di nuova regola operativa o di modifica a una procedura esistente, aprendo la relativa Issue^G.
3. La modifica alle Norme di Progetto^G segue le procedure del processo di documentazione, così che nessuna nuova regola entri in vigore senza essere stata verificata.
4. L'Amministratore^G misura nell'iterazione successiva l'effetto dell'iniziativa sulle metriche interessate e ne riporta l'esito nella Retrospective seguente.

Questo meccanismo garantisce che le osservazioni emerse dalle retrospettive non restino semplici constatazioni, ma si traducano in norme vincolanti per gli Sprint^G successivi.

3 Qualità del software

Con l'espressione "qualità del software" si indica la totalità delle proprietà di un'entità che concorrono a soddisfare le necessità degli utenti, siano esse dichiarate esplicitamente o sottintese. Tale valutazione si focalizza su due direttrici fondamentali: il valore del prodotto finito e l'efficacia del processo di realizzazione.

L'eccellenza del risultato finale scaturisce invariabilmente dal valore del percorso produttivo intrapreso. Per garantire un software di alto livello, la strategia di valutazione deve essere programmata con cura e non limitata ad una sola verifica finale: tale approccio favorisce una gestione proattiva basata sul monitoraggio costante, superando la logica reattiva volta alla semplice risoluzione delle anomalie.

L'intero processo di valutazione si articola in quattro fasi principali (definizione preventiva delle metriche, misurazione, valutazione dei dati e accettazione finale).

3.1 Qualità del prodotto

Per definire e misurare la qualità intrinseca del software, lo standard di riferimento adottato è la serie di norme ISO/IEC 9126. Questo standard internazionale organizza la qualità in due diverse categorie:

1. **Qualità del Prodotto**, che analizza le seguenti proprietà intrinseche del software:

- Funzionalità;
- Affidabilità;
- Efficienza;
- Usabilità;
- Mantenibilità;
- Portabilità.

2. **Qualità in Uso**, che valuta la capacità del software di permettere agli utenti di raggiungere i propri obiettivi con efficacia, produttività, sicurezza e soddisfazione in specifici contesti d'uso.

Al fine di ottenerne una misurazione oggettiva, sono state stabilite specifiche metriche di prodotto, le quali trovano puntuale rendicontazione e misurazione all'interno del *Piano di Qualifica*^G. Tali metriche consentono di analizzare in termini quantitativi elementi determinanti sia del codice sorgente che dell'applicativo finale, tra cui la funzionalità, l'affidabilità, l'efficienza, l'usabilità, la manutenibilità e la portabilità.

3.2 Qualità del processo

Lo standard di riferimento per la qualità del processo è l'ISO/IEC 12207. Questo standard fornisce le linee guida per la gestione della qualità nei processi di sviluppo, includendo la pianificazione^G, il controllo e il miglioramento continuo.

Secondo questo standard la qualità del processo si basa su diversi principi fondamentali:

- **Focalizzazione su Scopo e Risultati (Purpose and Outcomes):** Questo rappresenta il pilastro fondamentale. Per garantire la qualità, ogni fase (dall'approvigionamento alla creazione, fino alla verifica) deve avere finalità ben definite e generare esiti attesi che siano concreti, misurabili e verificabili.
- **Approccio Modulare (Modularity):** La struttura dei processi deve basarsi su componenti caratterizzate da "forte coesione e debole accoppiamento". Ogni singolo modulo deve concentrarsi su un unico traguardo ed essere autonomo, così da limitare le ripercussioni sull'intero ciclo produttivo in caso di variazioni o criticità in un'area specifica.
- **Attribuzione delle Responsabilità (Responsibility):** Seguendo il principio per cui una responsabilità indistinta equivale a nessuna responsabilità, lo standard richiede che ogni attività sia assegnata a un ruolo preciso (come Sviluppatore, Fornitore, Acquisitore o Verificatore^G), eliminando incertezze all'interno dell'organizzazione.
- **Personalizzazione dei Processi (Tailoring):** Data la diversità tra i vari progetti software, lo standard prevede il principio del Tailoring. Le aziende devono adattare i processi alle proprie esigenze, selezionando esclusivamente le procedure pertinenti alla complessità, al rischio^G e alle risorse disponibili, evitando carichi burocratici superflui.
- **Evoluzione Continua e Tracciabilità:** È essenziale che ogni intervento, anomalia o variazione sia riconducibile al requisito di partenza. Il sistema non è statico, poiché integra cicli di audit, revisione e gestione delle problematiche. Ciò permette al gruppo di lavoro di ottimizzare costantemente le proprie metodologie, avvalendosi spesso di modelli quali CMMI o SPICE per attestare il livello di maturità raggiunto.

3.2.0.1 Applicazione del Tailoring Lo standard ISO/IEC 12207:1997 viene adattato al contesto di un team accademico che sviluppa un MVP^G stand-alone con moduli AI^G. Sono mantenuti i processi di **Fornitura** e **Sviluppo** (adattati con Agile^G/SCRUM), mentre Acquisizione, Operazione e Manutenzione sono esclusi. La documentazione è ridotta ai soli artefatti richiesti dalla baseline; la gestione della configurazione è delegata a GitHub^G. L'efficacia del tailoring è monitorata tramite la metrica **MPC-QMS**: un valore sotto soglia segnala la necessità di rivedere le scelte nell'iterazione successiva.

In questo progetto, tale obiettivo viene perseguito attraverso una precisa delineazione di metodologie e strumenti operativi. Il controllo costante dell'avanzamento è garantito da sistemi di tracciamento dei problemi (*issue*^G *tracking*) e da incontri periodici formalizzati, supportati da un'analisi rigorosa dei requisiti e da verifiche regolari sugli standard qualitativi dei risultati ottenuti.

Al fine di assicurare un monitoraggio imparziale, il gruppo ha stabilito specifiche metriche di processo, dettagliate all'interno del Piano di Qualifica^G. Tali indicatori permettono di misurare l'efficienza del ciclo di sviluppo, focalizzandosi su variabili critiche quali la puntualità nelle consegne, l'aderenza al budget stabilito, la frequenza di anomalie rilevate, l'estensione dei test e la conformità complessiva ai criteri prefissati.

4 Metriche

Questa sezione definisce in modo formale tutte le metriche adottate dal gruppo per quantificare la qualità del processo di sviluppo e del prodotto software finale. I valori di soglia (accettabili e ottimali) e i risultati effettivi delle misurazioni sono documentati nel Piano di Qualifica^G, che contiene il cruscotto di valutazione con l'andamento storico delle misurazioni.

Ogni metrica è descritta in termini di:

- **Identificativo:** codice univoco della metrica;
- **Nome:** denominazione della metrica;
- **Descrizione:** definizione operativa dello scopo della metrica;
- **Formula:** espressione matematica per il calcolo del valore;
- **Unità di misura:** unità in cui il valore è espresso.

Le metriche di processo sono identificate dalla sigla **MPC** (Metrica di Processo e Controllo), seguita da un mnemonico che richiama il nome della metrica, mentre le metriche di prodotto sono identificate dalla sigla **MPD** (Metrica di Prodotto) seguita da un numero progressivo. Gli identificativi assegnati non devono essere modificati né riutilizzati, poiché sono richiamati dal cruscotto di valutazione del Piano di Qualifica^G e dal tracciamento dei requisiti.

Le metriche di processo sono raggruppate per processo misurato, secondo la medesima classificazione adottata nel capitolo sull'architettura dei processi. I processi dotati di metriche sono fornitura, sviluppo, documentazione, verifica, accertamento della qualità e gestione dei processi. I restanti processi normati, ovvero gestione della configurazione, validazione, infrastruttura, formazione e miglioramento dei processi, non hanno metriche dedicate: il loro andamento è osservato indirettamente attraverso le metriche dei processi che ne dipendono, in particolare tramite **MPC-QMS** per la conformità complessiva e **MPD01** per la copertura dei requisiti. Qualora una Sprint^G Retrospective rilevi un'inefficienza non osservabile con le metriche esistenti, la definizione di una nuova metrica è trattata come iniziativa di miglioramento secondo la procedura del processo di miglioramento dei processi.

4.1 Metriche per la qualità di processo

Queste metriche misurano l'efficacia, l'efficienza e il controllo delle attività necessarie per sviluppare il prodotto. Il riferimento metodologico adottato è lo standard ISO/IEC 12207, adattato al Way of Working^G del gruppo. Le metriche non valutano il prodotto finito, ma il modo in cui esso viene realizzato.

4.1.1 Processi primari — Fornitura

Le metriche seguenti misurano l'aderenza al preventivo^G e alle scadenze stabilite dall'attività di stima e pianificazione^G delle risorse del processo di fornitura. I dati di ingresso sono i preventivi^G e i consuntivi^G registrati dal Responsabile^G nel Piano di Progetto^G e le ore rilevate dal tracciamento delle ore.

4.1.1.1 MPC-EV — Earned Value

- **Descrizione:** Valore del lavoro effettivamente completato, misurato come la quota di valore pianificato corrispondente alle attività portate a termine nello sprint^G.

- **Formula:**

$$EV = PV \times \frac{\text{Ore Complesate}}{\text{Ore Pianificate}}$$

- **Unità di misura:** Euro

4.1.1.2 MPC-PV — Planned Value

- **Descrizione:** Valore economico del lavoro pianificato per lo sprint^G, estratto dai preventivi del Piano di Progetto^G. Rappresenta il costo previsto per le attività programmate.

- **Formula:**

$$PV = \sum_{i=1}^n (\text{Ore Previste}_i \times \text{Tariffa Oraria}_i)$$

dove n è il numero di task pianificate nello sprint^G e la tariffa oraria corrisponde al ruolo assegnato alla task.

- **Unità di misura:** Euro

4.1.1.3 MPC-AC — Actual Cost

- **Descrizione:** Costo effettivamente sostenuto per le ore lavorate nello sprint^G, ricavato dai consuntivi del Piano di Progetto^G.

- **Formula:**

$$AC = \sum_{i=1}^n (\text{Ore Effettive}_i \times \text{Tariffa Oraria}_i)$$

dove n è il numero di task completate nello sprint^G e la tariffa oraria corrisponde al ruolo assegnato alla task.

- **Unità di misura:** Euro

4.1.1.4 MPC-CPI — Cost Performance Index

- **Descrizione:** Misura l'efficienza dei costi del progetto. Un valore inferiore a 1 indica che il progetto sta spendendo più del valore prodotto; un valore pari a 1 indica perfetta efficienza di costo.

- **Formula:**

$$CPI = \frac{EV_{cum}}{AC_{cum}}$$

dove i valori cumulativi sommano tutti gli sprint^G dall'inizio del progetto.

- **Interpretazione:**

- $CPI > 1$: il progetto produce valore a un costo inferiore al previsto
- $CPI = 1$: il progetto è in linea con il budget
- $CPI < 1$: il progetto sta spendendo più del valore prodotto

- **Unità di misura:** adimensionale

4.1.1.5 MPC-SPI — Schedule Performance Index

- **Descrizione:** Misura l'efficienza della pianificazione^G temporale. Un valore inferiore a 1 indica ritardo rispetto al piano; un valore pari a 1 indica perfetto rispetto delle scadenze.

- **Formula:**

$$SPI = \frac{EV_{cum}}{PV_{cum}}$$

- **Interpretazione:**

- $SPI > 1$: il progetto è in anticipo rispetto al piano
- $SPI = 1$: il progetto è in linea con il piano
- $SPI < 1$: il progetto è in ritardo rispetto al piano

- **Unità di misura:** adimensionale

4.1.1.6 MPC-EAC — Estimated At Completion

- **Descrizione:** Stima del costo totale finale del progetto proiettando l'efficienza di costo corrente (CPI) sull'intero budget rimanente.

- **Formula:**

$$EAC = \frac{BAC}{CPI}$$

- **Interpretazione:**

- $EAC < BAC$: il progetto si concluderà sotto budget
- $EAC = BAC$: il progetto rispetterà il budget
- $EAC > BAC$: il progetto supererà il budget

- **Unità di misura:** Euro

4.1.1.7 MPC-ETC — Estimate To Complete

- **Descrizione:** Stima del costo rimanente necessario per completare il progetto a partire dallo sprint^G corrente, ottenuta sottraendo il costo cumulativo effettivo dalla stima a completamento.

- **Formula:**

$$ETC = EAC - AC_{cum}$$

dove AC_{cum} è la somma dei costi effettivi sostenuti dall'inizio del progetto fino allo sprint^G corrente.

- **Unità di misura:** Euro

4.1.1.8 MPC-TEAC — Time Estimate At Completion

- **Descrizione:** Stima della durata totale del progetto a partire dalla performance di avanzamento misurata dallo SPI. Se $SPI = 1$, il progetto terminerà esattamente nei tempi pianificati.

- **Formula:**

$$TEAC = \frac{\text{Durata pianificata}}{SPI}$$

- **Unità di misura:** settimane

4.1.2 Processi primari — Sviluppo

La metrica seguente misura la stabilità dell'esito dell'attività di analisi e tracciamento dei requisiti del processo di sviluppo. Il dato di ingresso è il numero di requisiti dell'Analisi dei Requisiti^G rispetto alla baseline^G precedente.

4.1.2.1 MPC-RSI — Requirements Stability Index

- **Descrizione:** Misura la stabilità dei requisiti nel tempo, confrontando il numero di requisiti modificati, eliminati o aggiunti rispetto alla baseline iniziale dell'Analisi dei Requisiti^G. Valori prossimi al 100% indicano un documento dei requisiti stabile e maturo.

- **Formula:**

$$RSI = \left(1 - \frac{N_{\text{modificati}} + N_{\text{eliminati}} + N_{\text{aggiunti}}}{N_{\text{totali iniziali}}} \right) \times 100\%$$

- **Unità di misura:** percentuale (%)

4.1.3 Processi di supporto — Documentazione

Le metriche seguenti misurano la qualità redazionale dei documenti prodotti dal processo di documentazione e sono rilevate dal Verificatore^G durante l'attività di verifica e approvazione, prima che il documento passi allo stato *Verificato*.

4.1.3.1 MPC-IG — Indice di Gulpease

- **Descrizione:** Indice di leggibilità calibrato per la lingua italiana. Valuta la complessità del testo in base alla lunghezza delle parole e delle frasi rispetto al numero totale di parole. Valori più alti indicano maggiore leggibilità. Il calcolo viene effettuato tramite estrazione del testo con detex^G e script Python.

- **Formula:**

$$G = 89 - \frac{10 \cdot L}{P} + \frac{300 \cdot F}{P}$$

dove L è il numero di lettere, P il numero di parole e F il numero di frasi.

- **Unità di misura:** adimensionale (scala 0–100)

4.1.3.2 MPC-CO — Correttezza Ortografica

- **Descrizione:** Numero di errori grammaticali o di battitura rilevati nei documenti tramite strumenti automatici di controllo ortografico. La verifica viene condotta tramite estrazione del testo con detex^G e analisi automatica. Il valore accettabile e ottimale è zero errori.

- **Formula:**

$$\text{CO} = \text{Numero di Errori Ortografici Rilevati}$$

- **Unità di misura:** numero intero (conteggio assoluto)

4.1.4 Processi di supporto — Verifica

Le metriche seguenti misurano l'estensione e l'esito dell'attività di analisi dinamica^G del processo di verifica. Sono rilevate automaticamente dalla pipeline di *Continuous Integration* a ogni *push* e riportate dal Programmatore^G nella descrizione della Pull Request^G.

4.1.4.1 MPC-CC — Code Coverage

- **Descrizione:** Percentuale di righe di codice sorgente effettivamente eseguite durante i test automatici. Misura il grado di copertura complessivo della suite di test. La metrica è applicabile a partire dalla fase PB^G, quando sarà disponibile il codice sorgente.

- **Formula:**

$$\text{CC} = \frac{\text{Linee di Codice Eseguite dai Test}}{\text{Linee di Codice Totali}} \times 100$$

- **Unità di misura:** percentuale (%)

4.1.4.2 MPC-TSR — Test Success Rate

- **Descrizione:** Percentuale di test automatici che hanno restituito un esito positivo sul totale dei test eseguiti. Un valore del 100% indica l'assenza di regressioni nella suite corrente.

- **Formula:**

$$\text{TSR} = \frac{\text{Test Superati}}{\text{Test Eseguiti}} \times 100$$

- **Unità di misura:** percentuale (%)

4.1.5 Processi di supporto — Accertamento della qualità

La metrica seguente sintetizza l'esito delle attività di accertamento della qualità di processo e di prodotto ed è rilevata dall'Amministratore^G al termine di ogni Sprint^G, come previsto dalla relativa procedura.

4.1.5.1 MPC-QMS — Quality Metrics Satisfied

- **Descrizione:** Percentuale di metriche di qualità (di processo e di prodotto) che rientrano nel range accettabile definito nel Piano di Qualifica^G. Rappresenta un indicatore sintetico della salute complessiva del progetto al termine di ciascuno sprint^G.

- **Formula:**

$$\text{QMS} = \frac{\text{Metriche soddisfatte}}{\text{Metriche misurate}} \times 100\%$$

- **Unità di misura:** percentuale (%)

4.1.6 Processi organizzativi — Gestione dei processi

Le metriche seguenti misurano l'efficienza nell'impiego delle risorse e l'efficacia della previsione dei rischi, ovvero gli esiti delle attività di pianificazione^G dell'iterazione e di revisione e valutazione del processo di gestione dei processi. I dati di ingresso sono il foglio di rendicontazione oraria e il Registro dei Rischi del Piano di Progetto^G.

4.1.6.1 MPC-TE — Time Efficiency

- **Descrizione:** Rapporto tra le ore dedicate ad attività produttive effettivamente rendicontabili e le ore totali disponibili nello sprint^G. Indica quanta parte del tempo disponibile si traduce direttamente in avanzamento del progetto.

- **Formula:**

$$\text{TE} = \frac{\text{Ore produttive}}{\text{Ore totali disponibili}} \times 100\%$$

- **Unità di misura:** percentuale (%)

4.1.6.2 MPC-PRI — Percentuale Rischi Inattesi

- **Descrizione:** Percentuale di rischi effettivamente manifestatisi che non erano stati identificati nel registro dei rischi^G del Piano di Progetto^G. Valori bassi indicano una pianificazione^G del rischio^G efficace e un registro dei rischi^G completo.

- **Formula:**

$$PRI = \frac{N_{\text{rischi inattesi}}}{N_{\text{rischi totali manifestati}}} \times 100\%$$

- **Unità di misura:** percentuale (%)

4.1.6.3 MPC-LE — Labor Efficiency

- **Descrizione:** Rapporto tra le ore dedicate ad attività a valore aggiunto (che producono direttamente un output di progetto) e il totale delle ore lavorate. Misura l'efficienza nell'impiego del lavoro del team, escludendo le attività non direttamente produttive.

- **Formula:**

$$LE = \frac{\text{Ore a valore aggiunto}}{\text{Ore totali lavorate}} \times 100\%$$

- **Unità di misura:** percentuale (%)

4.2 Metriche per la qualità di prodotto

Queste metriche misurano le proprietà interne ed esterne del software finale per garantire che il sistema rispetti i requisiti. Lo standard di riferimento è ISO/IEC 9126, adottato come base per la classificazione delle caratteristiche di qualità del prodotto.

I raggruppamenti seguenti corrispondono alle sei caratteristiche di qualità del prodotto elencate nel capitolo sulla qualità del software, ovvero funzionalità, affidabilità, efficienza, usabilità, mantenibilità e portabilità. Tutte le metriche di prodotto sono rilevate dal Verificatore^G nell'ambito dell'attività di accertamento della qualità di prodotto e nessuna modifica può essere approvata se porta una di esse sotto la soglia accettabile fissata nel Piano di Qualifica^G.

4.2.1 Funzionalità

4.2.1.1 MPD01 — Requisiti Obbligatori Soddisfatti

- **Descrizione:** Percentuale di requisiti obbligatori correttamente implementati e verificati rispetto al totale dei requisiti obbligatori rilevati nell'Analisi dei Requisiti^G. Un requisito è considerato soddisfatto quando il relativo test di sistema^G risulta superato.

- **Formula:**

$$MPD01 = \frac{\text{Requisiti Obbligatori Soddisfatti}}{\text{Requisiti Obbligatori Totali}} \times 100$$

- **Unità di misura:** percentuale (%)

4.2.1.2 MPD02 — Requisiti Opzionali Soddisfatti

- **Descrizione:** Percentuale di requisiti opzionali^G correttamente implementati e verificati rispetto al totale dei requisiti opzionali^G rilevati. Un requisito è considerato soddisfatto quando il relativo test di sistema^G risulta superato.

- **Formula:**

$$\text{MPD02} = \frac{\text{Requisiti Opzionali}^G \text{ Soddisfatti}}{\text{Requisiti Opzionali}^G \text{ Totali}} \times 100$$

- **Unità di misura:** percentuale (%)

4.2.1.3 MPD03 — AI Acceptance Rate

- **Descrizione:** Percentuale di generazioni AI^G valutate dagli utenti con un rating^G pari o superiore a 3 su 5 rispetto al totale delle generazioni valutate. Misura la qualità percepita delle risposte prodotte dai moduli AI^G Assistant e AI^G Co-Pilot del sistema.

- **Formula:**

$$\text{MPD03} = \frac{\text{Generazioni con } \text{rating}^G \geq 3/5}{\text{Generazioni totali valutate}} \times 100$$

- **Unità di misura:** percentuale (%)

4.2.2 Affidabilità

4.2.2.1 MPD04 — Branch Coverage

- **Descrizione:** Percentuale di rami decisionali (ad esempio i rami *true* e *false* di un costrutto *if*) eseguiti durante i test automatici. Garantisce che tutte le possibili direzioni del flusso logico siano state verificate dalla suite di test.

- **Formula:**

$$\text{MPD04} = \frac{\text{Rami Decisionali Eseguiti}}{\text{Rami Decisionali Totali}} \times 100$$

- **Unità di misura:** percentuale (%)

4.2.2.2 MPD05 — Defect Density

- **Descrizione:** Densità di difetti nel codice sorgente, calcolata come numero di difetti rilevati per ogni KLOC^G (migliaia di righe di codice). Valori bassi indicano maggiore qualità del codice prodotto e minore frequenza di anomalie.

- **Formula:**

$$\text{MPD05} = \frac{\text{Numero di Difetti Rilevati}}{\text{KLOC}^G}$$

dove KLOC^G = Righe di Codice Totali/1000.

- **Unità di misura:** difetti per KLOC^G

4.2.3 Efficienza

4.2.3.1 MPD06 — Latenza Interfaccia Utente

- **Descrizione:** Tempo di risposta del sistema a un'interazione dell'utente sull'interfaccia, misurato come intervallo tra l'invio della richiesta e la ricezione della risposta completa per le operazioni generali dell'applicazione (escluse le chiamate ai moduli AI^G).

- **Formula:**

$$\text{MPD06} = T_{\text{risposta}} - T_{\text{richiesta}}$$

- **Unità di misura:** secondi (s)

4.2.3.2 MPD07 — Latenza Generazione Testo AI Assistant

- **Descrizione:** Tempo intercorso tra l'invio del prompt^G al modulo AI Post Generator^G e la ricezione del testo della bozza^G completa generata dal servizio AI^G.

- **Formula:**

$$\text{MPD07} = T_{\text{risposta AI}^G} - T_{\text{invio prompt}^G}$$

- **Unità di misura:** secondi (s)

4.2.3.3 MPD08 — Latenza Generazione Immagine AI Assistant

- **Descrizione:** Tempo intercorso tra la richiesta di generazione dell'immagine di copertina al modulo AI^G Assistant e la ricezione dell'immagine generata e pronta per la visualizzazione.

- **Formula:**

$$\text{MPD08} = T_{\text{ricezione immagine}} - T_{\text{richiesta immagine}}$$

- **Unità di misura:** secondi (s)

4.2.3.4 MPD09 — Latenza Analisi Documentale AI Co-Pilot

- **Descrizione:** Tempo intercorso tra il caricamento di un documento e la restituzione dei metadati^G classificati dal modulo AI Analyst^G (classificazione tipologia, estrazione destinatario, livello di confidenza^G).

- **Formula:**

$$\text{MPD09} = T_{\text{risultato analisi}} - T_{\text{caricamento documento}}$$

- **Unità di misura:** secondi (s)

4.2.4 Usabilità

4.2.4.1 MPD10 — Profondità Massima di Navigazione

- **Descrizione:** Numero massimo di livelli gerarchici che un utente deve attraversare per raggiungere qualsiasi funzionalità del sistema a partire dalla schermata principale. Valori bassi indicano un'interfaccia piatta e facilmente accessibile.

- **Formula:**

$$\text{MPD10} = \max \{ \text{Livelli di navigazione necessari per raggiungere la funzionalità } f, \forall f \}$$

- **Unità di misura:** livelli di navigazione

4.2.4.2 MPD11 — Learning Time

- **Descrizione:** Tempo medio necessario a un utente privo di esperienza pregressa con il sistema per apprendere le funzionalità principali in modo autonomo, misurato tramite sessioni di user testing. La verifica avviene confrontando il tempo con la soglia accettabile di 30 minuti.

- **Formula:**

$$\text{MPD11} = \frac{\sum_{i=1}^n T_{\text{apprendimento}_i}}{n}$$

dove n è il numero di utenti coinvolti nella sessione di test e $T_{\text{apprendimento}_i}$ è il tempo impiegato dall'utente i per completare le funzionalità principali senza assistenza.

- **Unità di misura:** minuti

4.2.5 Mantenibilità

4.2.5.1 MPD12 — Code Smell

- **Descrizione:** Numero di code smell rilevati nel codice sorgente tramite analisi statica^G. I code smell sono indicatori di progettazione debole (metodi troppo lunghi, classi troppo grandi, duplicazione di codice) che, pur non essendo errori bloccanti, rendono il sistema fragile e difficile da mantenere nel tempo.

- **Formula:**

$$\text{MPD12} = \text{Numero di Code Smell Rilevati}$$

- **Unità di misura:** numero intero (conteggio assoluto)

4.2.5.2 MPD13 — Coefficient of Coupling

- **Descrizione:** Misura il grado di interdipendenza tra i moduli del software. Un accoppiamento elevato implica che una modifica in un modulo rischi di propagarsi ad altri (effetto a catena), riducendo la manutenibilità del sistema.

- **Formula:**

$$\text{MPD13} = \frac{\text{Dipendenze Effettive tra Moduli}}{n \times (n - 1)}$$

dove n è il numero totale di moduli del sistema e $n \times (n - 1)$ rappresenta il numero massimo di dipendenze direzionali possibili. Una dipendenza sussiste quando un modulo importa, invoca o referencia direttamente un altro modulo.

- **Interpretazione:**

- MPD13 = 0: nessun accoppiamento (moduli completamente indipendenti)
- MPD13 = 1: accoppiamento massimo (ogni modulo dipende da tutti gli altri)

- **Unità di misura:** adimensionale (scala 0–1)

4.2.5.3 MPD14 — Cyclomatic Complexity

- **Descrizione:** Misura la complessità logica del codice contando il numero di percorsi linearmente indipendenti attraverso il flusso di controllo (costrutti `if`, `switch`, cicli). Valori elevati indicano codice difficile da testare e comprendere. Il valore riportato nel cruscotto è riferito al singolo metodo.

- **Formula:**

$$M = E - N + 2P$$

dove E è il numero di archi nel grafo di controllo del flusso, N il numero di nodi (istruzioni) e P il numero di componenti connesse del grafo (solitamente $P = 1$ per una singola funzione).

- **Unità di misura:** numero intero (adimensionale)

4.2.6 Portabilità

4.2.6.1 MPD15 — Compatibilità Cross-Browser

- **Descrizione:** Percentuale di browser target (principali browser moderni evergreen nelle ultime due versioni stabili) sui quali il sistema funziona correttamente senza degradazione delle funzionalità o dell'interfaccia utente.

- **Formula:**

$$\text{MPD15} = \frac{\text{Browser Supportati correttamente}}{\text{Browser Target Totali}} \times 100$$

- **Unità di misura:** percentuale (%)